Design and Development of an Autonomous Cooking System

Submitted in partial fulfillment of the requirements

of the degree of

Bachelor of Technology (B.Tech)

By
Kshitika B. Gala
(Roll No. M1810012)
Manav M. Mepani
(Roll No. M1810029)
Tanish A. Mishra
(Roll No. M1810034)

Supervisor:

Dr. Kiran S. Bhole

Department of Mechanical Engineering

Sardar Patel College of Engineering

2022

CERTIFICATE

This is to certify that the project entitled **"Design and Development of an Autonomous Cooking System"** is a bonafide work of **Kshitika B. Gala, Manav M. Mepani, Tanish A. Mishra (Roll No. M1810012, M1810029, M1810034)** submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of **B. Tech** in **Mechanical Engineering**.

(Name and sign)
Supervisor/Guide

(Name and sign)
Co-Supervisor/Co-Guide

(Name and Sign)
Head of Department

(Name and Sign)
Principal

Project Report Approval for B. Tech.

This project report entitled "Design and Development of an Autonomous Cooking System" by Kshitika B. Gala, Manav M. Mepani and Tanish A. Mishra is approved for the degree of B. Tech in Mechanical Engineering

Examiners

1.----------

2.----------

Date:

Place:

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Kshitika B. Gala
M1810012

Manav M. Mepani
M1810029

Tanish A. Mishra
M1810034

Date:

Place: SPCE, Mumbai - 400058

Index

Acknowledgments 5

Abstract 6

Chapter 1. Introduction 7

1.1 Problem Statement & Solution 7

1.2 Project Summary

Chapter 2. Literature Review & Market Research 7

2.2 Literature Review 7

2.2 Existing Models Review 8

2.3 Market Research 12

Chapter 3. Design 13

3.1 Design goals 13

3.2 Methodology 13

3.3 Preliminary Functional Design 14

3.4 Detail design 15

3.5 Interaction with Industry Personnel 26

3.6 Design Modification for using Standard Parts 27

Chapter 4. Analysis & Simulation 28

4.1 Structural Analysis 28

4.2 Thermal Analysis 30

4.3 Simulation & Kinematic Analysis 33

Chapter 5. Fabrication & Testing 37

5.1 Material Selection 37

5.2 Manufacturing Methodology 38

5.3 Standard Parts 40

5.4 Electronics 41

5.5 The Code 44

5.6 Testing 56

Chapter 6. Results & Discussion 58

6.1 Results & Conclusion 58

6.2 Future Scope 59

Chapter 7. References 60

Acknowledgements

We would like to first thank Sardar Patel College of Engineering, Mumbai for the knowledge & constant support they have provided at all levels. Next, we would like to thank our guide Dr. Kiran S. Bhole who assisted us whenever we needed it and made the entire project process seamless. We thank the SPCE Robocon team for providing us with space and equipment required to manufacture our project. We are deeply grateful to Dr. N. Raykar, for providing feedback & suggestions which helped during the design phase along with Mr. Sanjiv Sarnaik and his associates at Prasanna Packaging Machinery Pvt. Ltd., for providing industrial feedback and future manufacturing & fabrication support. We would also like to thank Prof. Jayram Gholave for providing Chemistry lab equipment for experimental validation of Thermal Analysis. We thank our co-guide Dr. Sangeeta Daingade from the Department of Electrical Engineering for her support and knowledge about the electronics and control system that was incorporated in the design.

Abstract

Robotic arms, by virtue of their versatility, find extensive usage in industry-level applications; from mechanical operations like welding and assembling to painting and sorting. Capitalizing on their relative absence in the household domain, and the increasing pervasiveness of domestic automation, this project seeks to develop a robot arm designed for carrying out operations involved in independently cooking a wide variety of food. The design, which aims to minimize cost while maximizing versatility; has been explained thoroughly, including choice of materials, degrees of freedom, actuators, linkage cross-sections, and sensors. The purview of necessary elementary motions is narrowed down by targeting specific recipes that the arm will be expected to cook. Further, a mechanism for dispensing appropriately proportioned ingredients has also been described, including bulk matter like vegetables as well as minor ingredients like spices and seasoning. Stress-Deformation Analysis has been carried out for verification of structural integrity of the most crucial components in the structure of the arm. The ultimate objective of this project is to develop a proof of concept for a household cooking robot by building on existing research in the domain.

Chapter 1. **Introduction**

1.1. Problem Statement

1.1.1. Motivation

Good nutrition is a basic human necessity. With the rise in lifestyle diseases, humans are now looking for healthy options as opposed to fast food. Non-home cooked food has always been less healthy and more expensive in comparison to home cooked food, making the latter a necessity for proper and affordable nutrition. But cooking is an everyday activity that demands a substantial amount of time and effort, a commitment that is becoming increasingly infeasible in the competitive environment of the modern world. Skipping breakfast, a hurried 15-minute lunch, and a late-night takeout dinner is increasingly becoming the norm for the working class. Students who migrate to different cities for college also face the same problem.

Advancements in various technological domains during the last two decades have transformed 'fictional' robots into reality. Robotics lies in the category of industrial automation. Pressing demands of enhanced productivity have necessitated deployment of robots to automate tasks. Today, robots are considered an integral part of industries. Historically, the population of industrial robots followed an increasing trend with the last year setting a new sales record. Food manufacturing and processing factories are now using cost effective automation solutions for higher production volume as compared to conventional processes. As the reliance on manual labor is considered a classical concept now, more preference is given to robotized handling/manufacturing installation. Common examples include; picking, placing, packaging and palletizing applications. Robotics has steadily been revolutionizing our industries by saving both human time and effort, it holds potential to do the same for our kitchen. Along with Industrial Robotics, Automation at home is also becoming popular. There are different machines to assist humans in our chores but nothing to cook food. The project aims to solve this problem through the use of an automated robotic system.

1.1.2. Solution

The solution here is to mimic the functions done by a human hand during cooking like stirring, mashing & flipping. A robotic manipulator can be used with a cutomised tool holder to carry out these tasks. The tools held by the tool holder are also specially designed to serve its purpose. A 6 DOF Manipulator with a PUMA Configuration was chosen because of its dexterity. Another part of the solution is the ingredient dispenser. Any basic cooking involves two things: dispensing ingredients & mixing them with appropriate temperature & time. An ingredient dispenser was designed which directly dispenses ingredients as well as spices in the

pan which can be heated by an Induction stove. It used a lead screw mechanism to open the lid and let the ingredients slide into the pan. Some other items like the liquid dispenser are also added which can be controlled by an electronically actuated valve. This liquid dispenser can be used for liquid ingredients or oil or liquified butter which is often used in cooking.

1.2. Project Summary

1.2.1. Project Name & Definition

The project **Design & Development of Automatic Cooking System** involves a 6 DOF robot arm to mimic human hand gestures during cooking and an ingredient dispenser mechanism to add ingredients to the pan.

1.2.2. Project Scope

The scope of this project spans from brainstorming to conceptual design, detailed CAD design and then to fabrication, simulation & testing.

- **Brainstorming:**

Different available literature & designs already in the market was studied closely. After which, similar mechanism related papers were also studied to get started on with the design.

- **Conceptualization:**

Initially a pen & paper design was made taking an Indian kitchen platform space into account. The different components of the design were placed to increase compactness as well as versatility. It included a robotic manipulator keeping it in a place where it can reach all items it has to with minimum movement. Then the induction stove & ingredients dispensers were placed.

After the functional design was finalised, CAD modeling was begun. First the robotic manipulator was designed keeping in mind torque reduction required to reduce load on the motors. A simulated annealing optimization was done to determine link lengths to reduce torque. After the manipulator, the solid ingredient dispenser was designed to fit in ingredients of 8 types in quantity enough for four people. In order to make things easier we selected three recipes with varying levels of difficulty - Bhindi Masala, Pav Bhaji & Dosa. After that, the oil & dosa batter dispenser was made.

- **Selection of Materials & Manufacturing Techniques:**

Once the design was made, each part was assigned materials taking into consideration temperature resistance, strength & manufacturability. Manufacturing techniques were also finalised.

- **Structural & Thermal Analysis:**

Structural Analysis on Ansys was done to make sure that the links don't crack under fatigue. Thermal Analysis was also done on Ansys Transient Thermal which was validated by an experiment. Thermal Analysis was done so that it could be checked whether the motors are getting heated or not.

- **Virtual Simulation:**

Motion Simulation was done to check the working of the manipulator and the inverse kinematics code.

- **Manufacturing of a Scaled Down Version of the system:**

The manufacturing of the entire design, especially the manipulator was done.

- **Validation & Testing of the Prototype for Proof of Concept**

Testing of the electronic circuit, motor testing, inverse kinematic code testing was done on the manipulator to conclude that it could do actions needed to cook dishes.

1.2.3. Benefits - Research

- Proposes an alternate approach to a automatic cooking system which can be further developed into an ergonomic & economic design
- Adds to the literature for reference regarding application of a robot arm in a domestic environment

1.2.4. Benefits - Corporate

- Proposes an alternate economical & versatile design, designed specifically for Indian households
- Opens up new applications of robotics in the industry
- Encouraging New Business Models in Food-Tech Industry

Chapter 2. **Literature Review & Market Research**

2.1. Literature Review

This venture into domestic cooking automation is not entirely unprecedented. Li et al [3] proposed a system to effectively cook Chinese recipes using a specialized 3 DOF mechanism that manipulates the cooking vessel. Start-ups like Nosh [4], Nymble [5], Mechanical Chef [6], etc also possess automated cooking systems but remain inaccessible due to their high cost. Further, such existing systems, for a lack of redundant degrees of freedom, cannot effectively carry out all actions necessary for cooking a large variety of dishes the way a human being can. [7-12]

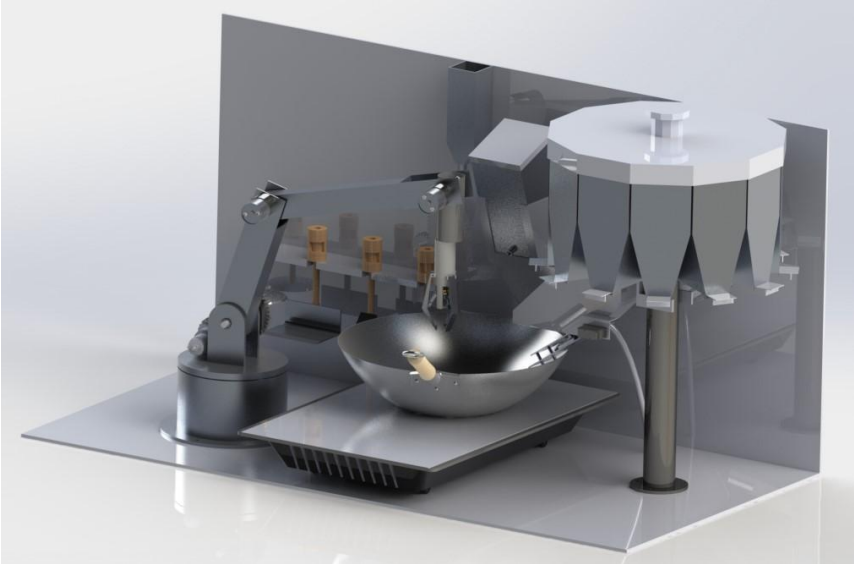


Fig. 1. CAD model of the proposed system with all its components.

Noh et al [9], in their design, employ 2 robotic arms that mimic human movement which has resulted in extremely high versatility. Their system, however, takes up considerable space and is difficult to accommodate in an average household kitchen. Additionally, due to the large number of sophisticated actuators it requires, the associated costs are high. Hence, this project hopes to achieve a compromise by using a single serial manipulator, trading some maneuverability for significantly reduced costs and space. Fig. 1 denotes the entire system with all its components in their appropriate relative positions.

2.2. Existing Models Review

While studying and reviewing several research papers we also came across some commercial models that were available in the market at a prototype stage. Some of these models were designed by Indian Start-ups and some available outside India.

2.2.1. Nosh (Euphotic Labs)

Nosh looks like a small oven which has an ingredient & spice tray, water & oil container, chimney / exhaust filter, a pan and a custom made stirrer. It is a smart machine which can cook upto 120 pre-coded varieties on one-pot meals. It is designed such that it is fully recyclable. Nosh is also equipped with an IOT system which enables the user to order the bot to cook the food via a mobile device and the user can see the process via live streaming. Fig. 2 shows the Nosh model created by an Indian Start-up.

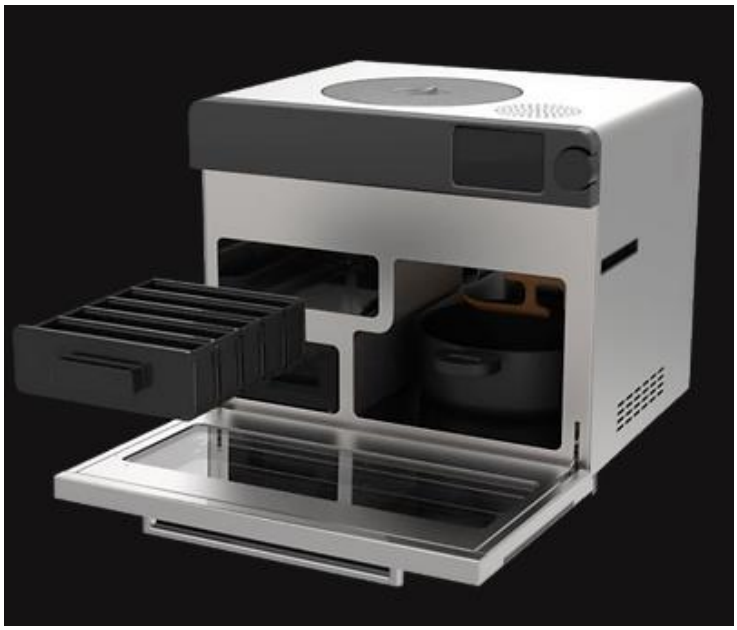


Fig. 2. Model of commercial product Nosh

Nosh uses a sliding dispensing mechanism for ingredients and using a time based algorithm, ingredients are dispensed and mixed. Here is a limitation of nosh based on its versatility, it can only perform one action of mixing with only 1DOF.

2.2.2. Nymble

Nymble's Julia robot, also called the unmanned kitchen arsenal, is an AI-powered kitchen robot. From measuring ingredient weights to making sure the right amount gets dispensed, to stirring the food with the dexterity of a human chef, it can do it all. As seen in Fig. 3 there are separate spice & ingredient trays, in-buit load cells & induction cooktop to maintain the temperature and weight of the ingredients, a customised 3DOF stirrer which can cook one-pot meals easily.

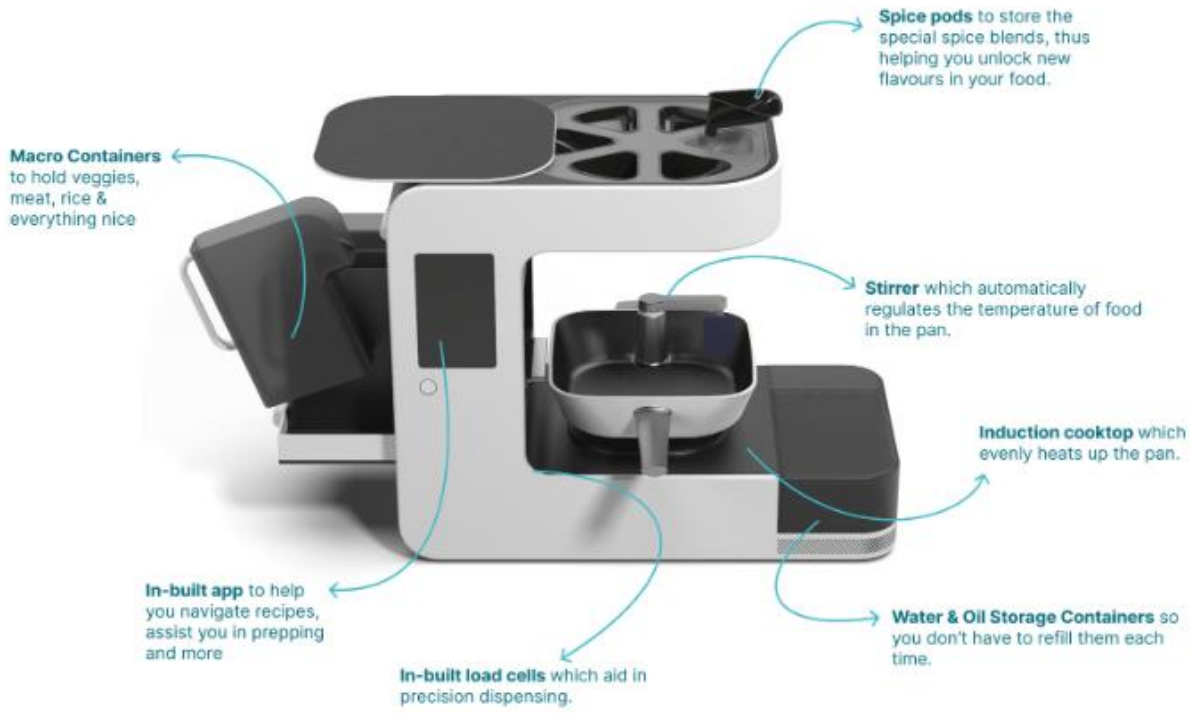


Fig. 3. Model of commercial product Nymble

It is also equipped with a chef-eye i.e. a camera which monitors if the food is properly cooked or not. The camera sees the change in color of the food, texture, and the gravies thicken. The AI system here is used to change the spice, saucy and amount of vegetable levels in every recipe thus customising as per the user. It has a store of 500+ recipes categorised based on diet (vegan, non-veg, etc), meal (breakfast,lunch, etc), and cuisine (thai, mediterranean, mexican ect).

2.2.3. Mechanical Chef

Mechanical Chef's robot is a 3 pan machine which can cook over three dishes together. It can prepare customized recipes and can be integrated with mobile as well. The design is as shown in Fig. 4 below.

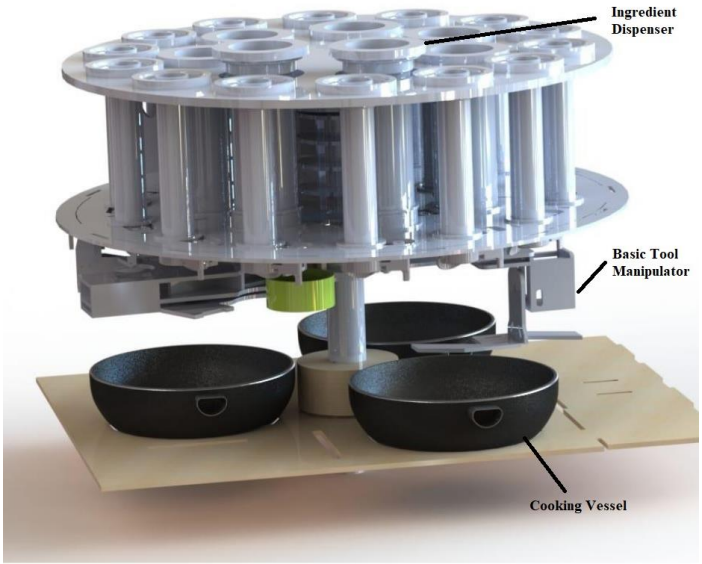


Fig. 4. Model of commercial product Mechanical Chef

2.2.4. Miso Robotics

Miso robotics' Flippy 2 (Fig. 5) has more than 120 configurations built into its technology and is the only robotic fry station currently being produced at scale. It has the capability of storage & dispensing, frying & drying, mixing & finishing.

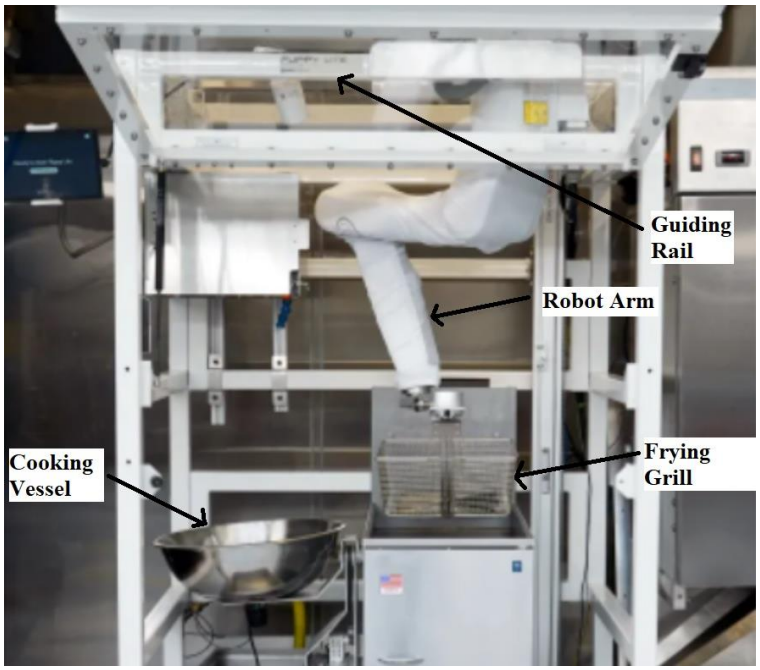


Fig. 5. Flippy by Miso Robotics (Physical Model)

3.3. Market Research

Robots essentially have the potential to transform the processes in food processing and handling, palletizing and packaging and food serving. Therefore, recent years have witnessed a tremendously increased trend of robots deployment in the food sector. Looking at it from a commercial perspective the Primary Target Market for this product is the Working Class who either don't like to or don't have time to cook food. In India

alone, about 67% of the population is of the working-age which is about 900 million people. This is going to give birth to an entirely new segment in the Consumer Electronics Industry. According to Research Nester, the Global Cooking Robot Market will grow by a CAGR of 16.09% from 2020 to 2028.

Chapter 3. **Design**

3.1. Design goals

A few Design Objectives were laid down before the start of the designing process.

- ❖ The system must be able to carry out elementary cooking actions like stirring, flipping, mashing.
- ❖ Must be able to cook using entirely raw, pre-sliced ingredients.
- ❖ Human intervention must be minimized and versatility must be maximized.
- ❖ Complexity of design must be minimized.

3.2. Methodology

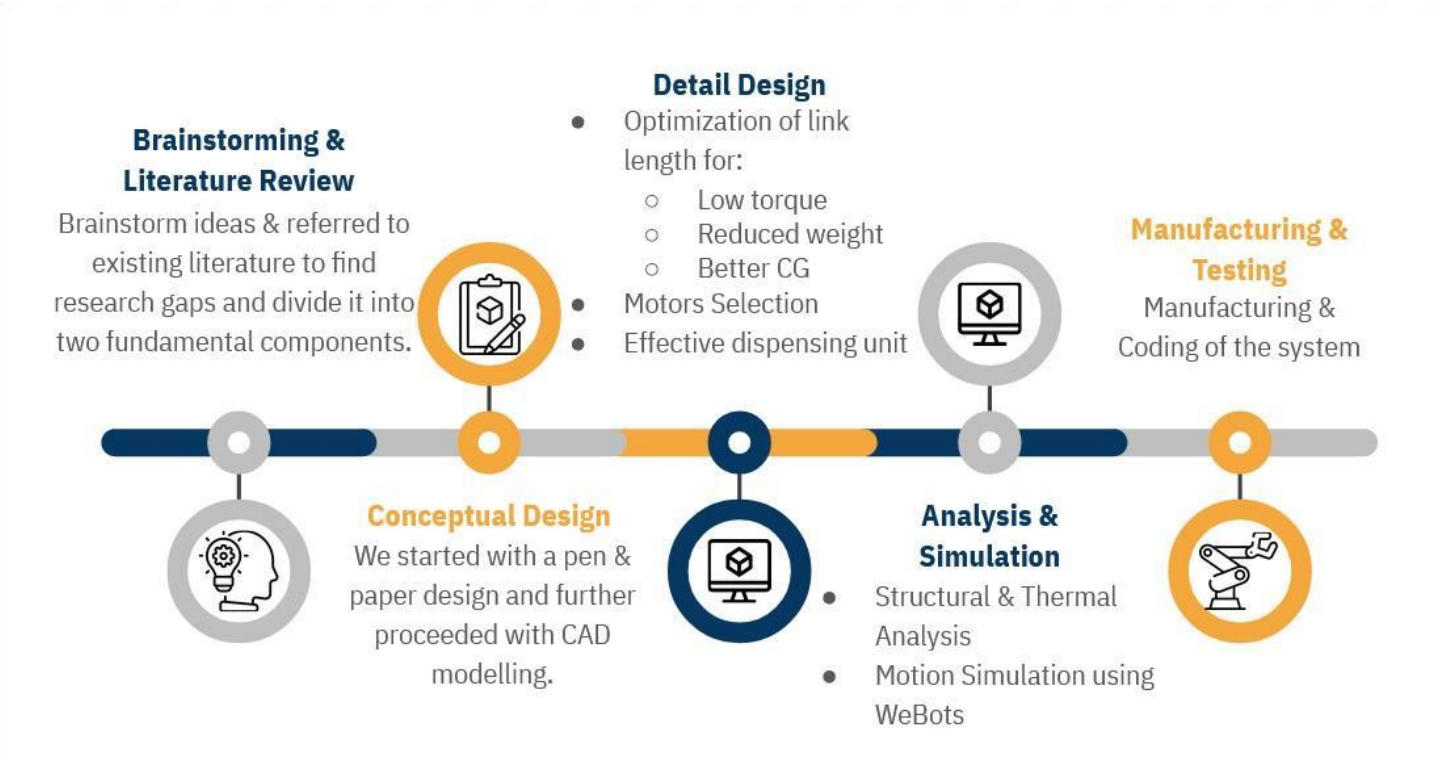


Fig. 6. Illustrative flowchart of the Methodology

- ❖ Through the literature review, it was found that the dual arm manipulator by Noh et al at UCLA possesses very high versatility but has a commensurately high cost due to a large number of actuators while taking up a lot of space. A single actuator costs tens of thousands of rupees. On the other hand, this chinese dish making design by Li et al is optimized only for flipping and tossing actions but is relatively affordable. Thus, there is room for innovation somewhere in between.
- ❖ By this it became clear that cost and versatility are directly proportional.
- ❖ We divided our design into 2 main parts: Robotic arm & dispensing mechanism.

- We decided to go with a 6 DOF robotic arm to have sufficient dexterity while trying to keep it low cost.
- Different containers and dispensing mechanisms for liquid & solid ingredients were designed.
- A preliminary pen & paper design was created to adjust all components
- Once we had a clear idea of what we need and what will be placed where, we started CAD modeling for detail designing.
- Using a simulated annealing code for optimization we optimized the link lengths to minimize torque.
- We later scaled down the model for one person and also replaced some of the custom parts with available standard parts for ease of prototyping.

3.3. Preliminary Functional Design

We first made a hand drawn sketch of our conceptual design as seen in Fig. 7 to visualize the general placements and structure of things we needed. In order to set proper design goals to achieve we selected three recipes with varying levels of difficulty - *Bhindi Masala, Pav Bhaji & Dosa*. Based on the nature of the functions needed to make selected recipes, we selected basic components & mechanisms and generated a block diagram of the whole system & finalised maximum dimension.

The general design contained oil, water, dosa and solid ingredients dispenser that are used within the average Indian Household kitchen. A single Robotic arm initially equipped with a tool locking mechanism as an end effector was decided to be used to replicate the functions of a human hand. A tool holder from where the robotic arm could access different tools.

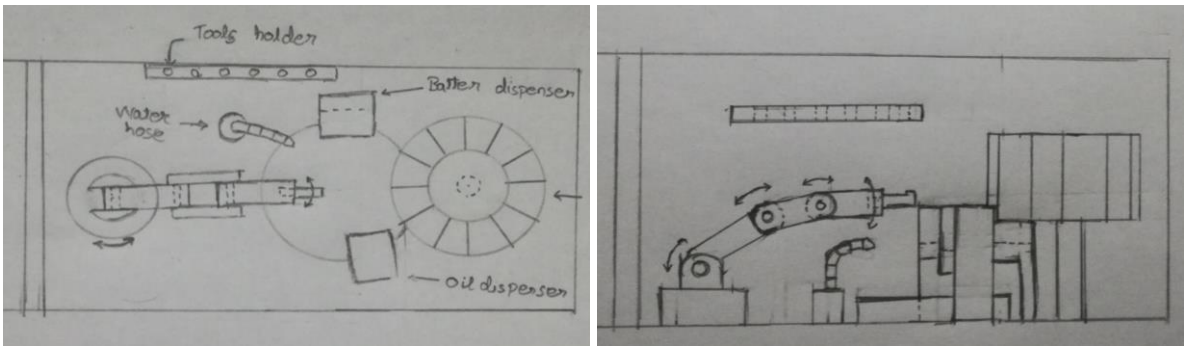
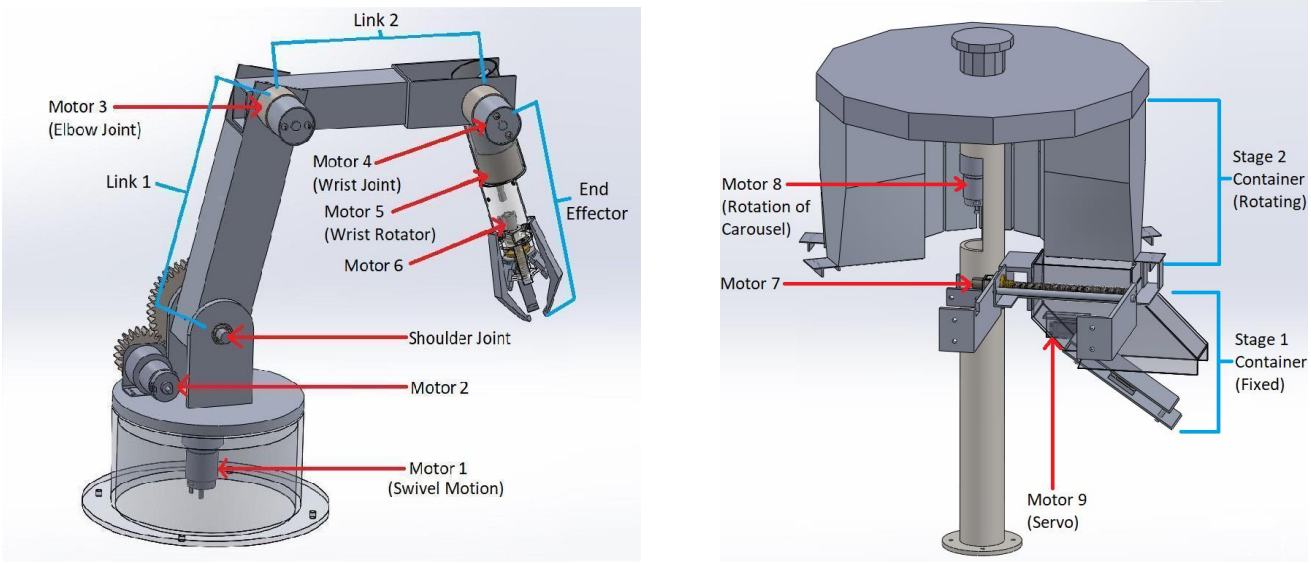


Fig. 7. Preliminary Functional Design

3.4. Detail design

The system can be divided into two major parts: the robot arm and the solid dispenser. Smaller dispensers for liquids and semi liquids are also present. The arm is tasked with actually carrying out the cooking actions while the dispensers, with dispensing the ingredients in appropriate quantities at appropriate times. These are placed in fixed predetermined positions relative to an induction cooktop and tool rack.



(a) (b)

Fig. 8. (a) Robot arm with important parts labeled (left); (b) Solid ingredient dispenser with important parts labeled (right)

A labeled diagram of both can be seen in Fig. 8 (a) & 8 (b) respectively. Dispensers are of two types: solid dispensers and liquid and semi-liquid dispensers. The specifications were decided based on the dishes being targeted and their quantity. Components are sized to serve 2 people and the maneuvers that the arm is expected to perform are chosen to cook the following 3 Indian dishes: "*Bhindi Masala, Pav Bhaji, and Dosa*".

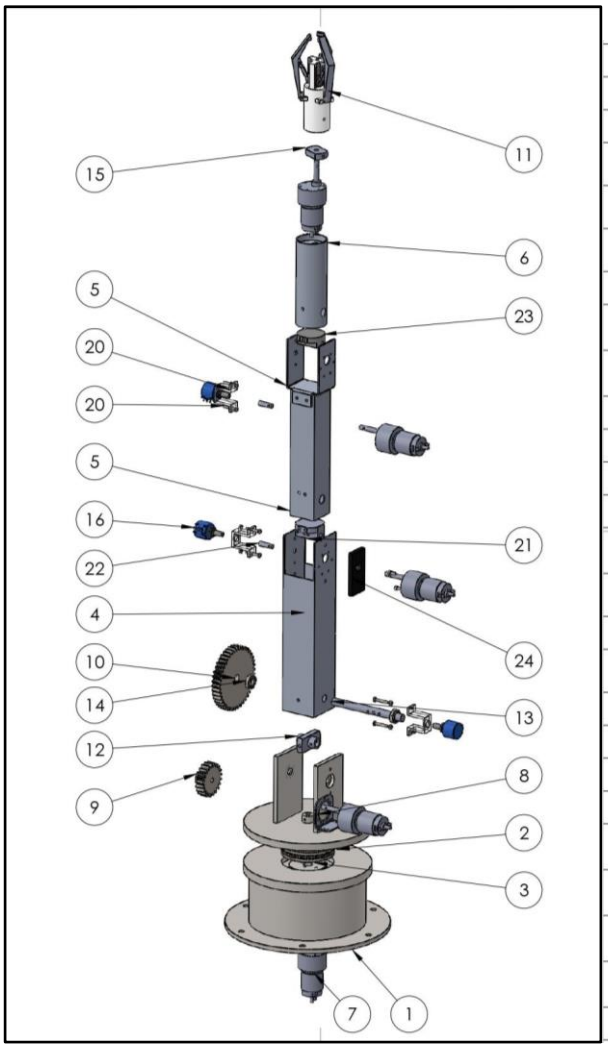
3.4.1. Robot Arm

Based on consideration of the required movements, the arm is designed with 5 rotary DOF (with an additional DOF for representing the position of claw).

The motor attached to the base (Motor 1) imparts a swivel motion to the arm. The other motors are at joints with positions corresponding to a human shoulder (Motor 2), elbow (Motor 3), and wrist (Motor 4). The end effector has an additional rotary degree of freedom about its axis (Motor 5) not present in a human, to optimize stirring action during the cooking.

These joints are sufficient for attaining the trajectories required for cooking the three recipes mentioned above, namely, mashing, stirring, and turning over ingredients in the pan (dosa in particular).

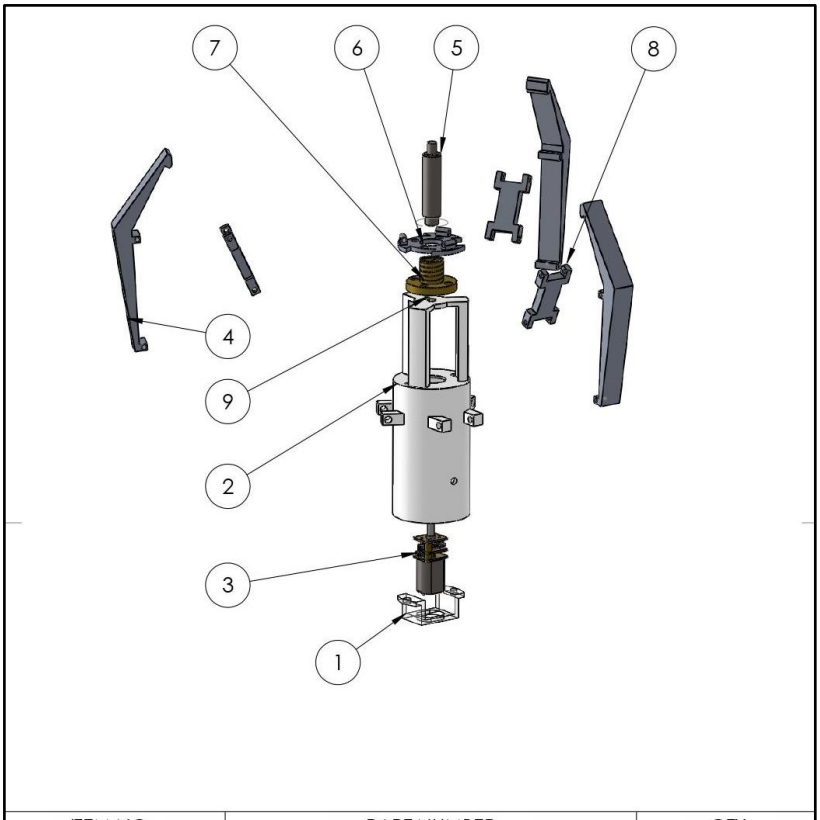
The exploded view for the same along with the BOM can be seen in the Fig. 9 given on the next page.



ITEM NO.	PART NUMBER	QTY.
1	Base of the bot (S)	1
2	Rotating base (S)	1
3	SKF - 51110 - 20,DE,AC,20_68	1
4	First arm (Scaled) (Aluminium)	1
5	Second arm (Scaled) (Aluminium)	1
6	Motor house (S)	1
7	johnson geard12v 10rpm dc motor	5
8	johnson-mini-johnson-gear-motor	1
9	Gear 1 (S)	1
10	Gear 2 (S)	1
11	CLaw 2.0	1
12	Motor coupler 1 (S)	1
13	Motor 1 coupler shaft (S)	1
14	SKF - 61800 - 14,DE,AC,14_68	2
15	Motor coupler 4 (S)	1
16	Potentiometer - Multiturn 01 v3	2
17	Pot holder	2
18	M3 nut	4
19	M3x20	4
20	Encoder assembly	1
21	Motor coupler 2 (S) (New)	1
22	Motor coupler shaft (S)	2
23	Motor coupler 3 (S) (New)	1
24	2nd link assembly plate	1
25	M3x8	2

Fig. 9. Exploded View of Robotic Arm assembly

Initially a collet or lock and key mechanism was considered as an end effector for the arm to provide high degree of locking between the end effector and the custom tools but later on a three fingered claw was used because the precision required in gripping is lower than that of a collet or lock and key mechanism. The claw assembly is seen in the Fig. 10 below

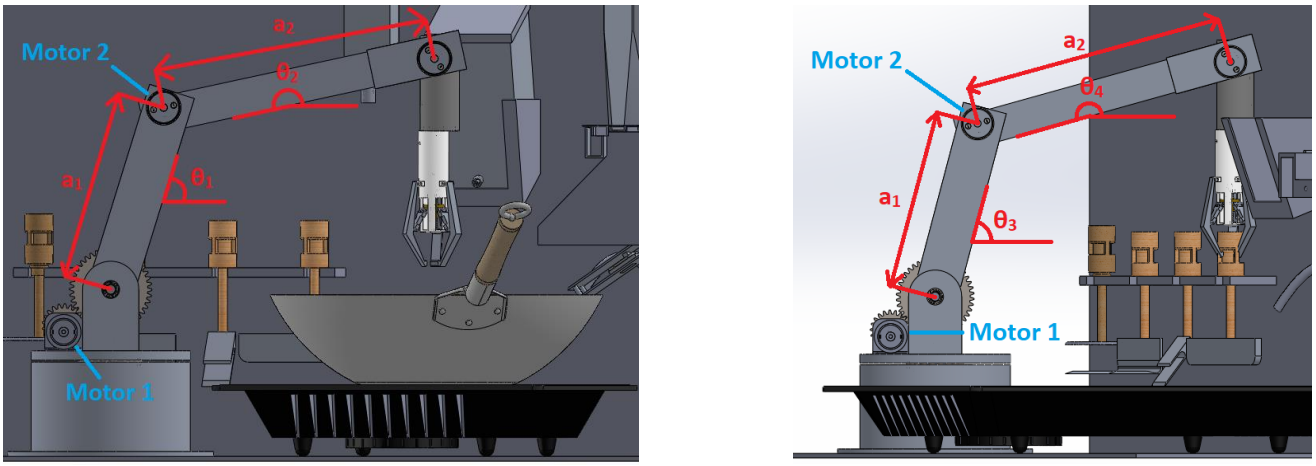


ITEM NO.	PART NUMBER	QTY.
1	Motor holder (S)	1
2	Motor house 2 (S)	1
3	Motor Assembly	1
4	Finger (S)	3
5	8mm trapez mil (32mm length) (S)	1
6	Brass nut attachment (S)	1
7	LeadScrew Nut 8mm x 2mmPitch (S)	1
8	Link (S)	3
9	M2x2 grub screw	1

Fig. 10. Exploded View of Claw assembly

3.4.1.1. Link Length Optimization

Lengths of individual links of the robot arm were chosen to minimize torque requirement at extreme positions of the arm. 2 Positions were chosen as shown in Fig. 11, as optimization for one position led to only trivial solutions. The method used for optimization was Simulated Annealing.



(a) (b)

Fig. 11. (a) Robot arm in position 1 (poised to act on the cooking vessel) (left); (b) Robot arm in position 2 (Poised to grab the furthest tool) (right)

The function being minimized is:

Max {$Torque_{11}$, $Torque_{12}$, $Torque_{21}$, $Torque_{22}$}

Where,

$$Torque_{11}=\frac{\lambda_1 * {a_1}^2 * cos\,(\theta_1)}{2} + M * a_1 * cos\,(\theta_1) + (a_1 * cos\,(\theta_1) - 0.5 * a_2 * cos\,(\theta_2)) * \lambda_2 * a_2 + (3 * M + E) * (a_1 * cos\,(\theta_1) - a_2 * cos\,(\theta_2))$$

.....(1)

$$Torque_{12}= \frac{-\lambda_2 * a_2^2 * cos\,(\theta_2)}{2} - (3 * M + E) * a_2 * cos\,(\theta_2)$$

.....(2)

$$Torque_{21}=\frac{\lambda_1 * {a_1}^2 * cos\,(\theta_3)}{2} + M * a_1 * cos\,(\theta_3) + (a_1 * cos\,(\theta_3) - 0.5 * a_2 * cos\,(\theta_4)) * \lambda_2 * a_2 + (3 * M + E) * (a_1 * cos\,(\theta_3) - a_2 * cos\,(\theta_4))$$

.....(3)

$$Torque_{22} = \frac{-\lambda_2 * a_2^2 * cos\,(\theta_4)}{2} - (3 * M + E) * a_2 * cos\,(\theta_4)$$

.....(4)

The 4 equations, of which the maximum is being chosen, represent the torque on motors 1 and 2 in positions 1 and 2 respectively.

Where,

$Torque_{ij}$ = torque at position i on joint j

λ_i=Linear Mass Density of Link i (i=1,2)

a_i=Length of Link i (i=1,2)

θ_i=Angle between the horizontal and link i while positioned above the cooking vessel (i=1,2)

θ_i=Angle between the horizontal and link i while positioned above the tool (i=3,4)

M=Mass of motors

E=Mass of End Effector Assembly

The equation consists of terms for the torque, along with a penalty term to disincentivize longer link lengths.

Subject to constraints:

$\left(a_1 * cos(\theta_1) - a_2 * cos\ (\theta_2)\right) < 28$ [Necessary Horizontal Span to reach the cooking vessel] …..(5)

$\left(a_1 * sin\ (\theta_3)\ - a_2 * sin\ (\theta_4)\right) < 28$ [Necessary Vertical Span to reach the tool] …..(6)

$\left(a_1 * cos\ (\theta_3)\ - a_2 * cos\ (\theta_4)\right) < 30$ [Necessary horizontal span to reach the tool] …..(7)

$0 < a_1 < 25$ [Limiting link length due to space constraints] …..(8)

$0 < a_2 < 25$ [Limiting link length due to space constraints] …..(9)

$30° < \theta_1 < 135°$ [Limiting range of angles to a reasonable interval] .....(10)

$160° < \theta_2 < 270°$ [Limiting range of angles to a reasonable interval] .....(11)

$30° < \theta_3 < 135°$ [Limiting range of angles to a reasonable interval] .....(12)

$90° < \theta_4 < 270°$ [Limiting range of angles to a reasonable interval] .....(13)

3.4.1.2. Torque Estimation

For the estimation of maximum torque, the arm is taken to be in the outstretched horizontal position [6,8,9]. A payload of 0.6 kg is assumed at the end, considering that the weight of the heaviest tool is roughly 0.2 kg and then multiplying by a safety factor of 3 to account for any additional forces.

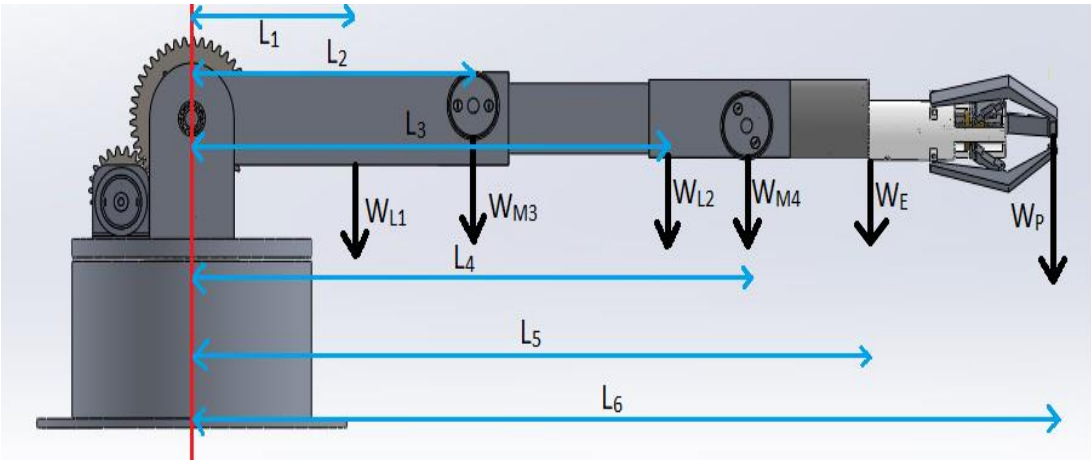


Fig. 12. Robot arm in a horizontally outstretched position

Based on Fig. 12:

$\tau_{net}(Torque\ at\ shoulder\ joint) = W_{L1} * L_1 + W_{M3} * L_2 + W_{L2} * L_3 + W_{M4} * L_4 + W_E * L_5 + 9.81 * L_6$ …..(14)

$\tau_1(Torque\ at\ elbow\ joint) = W_{L2} * (L_3 - L_2) + W_{M4} * (L_4 - L_2) + W_E * (L_5 - L_2) + 9.81 * (L_6 - L_2)$ …..(15)

$\tau_2(Torque\ at\ wrist\ joint) = W_E * (L_5 - L_4) + 9.81 * (L_6 - L_4)$ …..(16)

Variables, their meaning, and their values (Obtained from the SolidWorks Model) are tabulated in Table 1.

Table 1. Variables of the Torque Equation & their Values

Variable	Explanation	Value
L_1	Distance Between Shoulder Joint and Center of Mass of Link 1	09.19 cm
L_2	Distance Between Shoulder Joint and Center of Mass of Motor 3	20.00 cm
L_3	Distance Between Shoulder Joint and Center of Mass of Link 2	28.74 cm
L_4	Distance Between Shoulder Joint and Center of Mass of Motor 4	39.42 cm
L_5	Distance Between Shoulder Joint and Center of Mass of End Effector Assembly	48.29 cm
L_6	Distance Between Shoulder Joint and Assumed Payload	61.50 cm
W_{L1}	Weight of Link 1	0.93 N
W_{L2}	Weight of Link 2	0.70 N
W_{M3}	Weight of Motor 3	1.67 N
W_{M4}	Weight of Motor 4	1.67 N
W_E	Weight of End Effector	2.63 N
W_P	Weight of Payload	5.89 N

Substituting these values appropriately,

$$\tau_{net}(Torque\ at\ shoulder\ joint) = 616.89Ncm$$

$$\tau_1(Torque\ at\ elbow\ joint) = 357.22Ncm$$

$$\tau_2(Torque\ at\ wrist\ joint) = 239.93Ncm$$

3.4.2. Dispensers

Dispenser assemblies are tasked with the deployment of ingredients into the cooking vessel at appropriate times and in appropriate quantities. Based on the type of ingredient being dispensed, the dispensers are divided into 3 types: Solid, Liquid, and Semi-Liquid Dispensers.

3.4.2.1. Solid Ingredient Dispenser

The solid dispenser is a carousel-type two-stage mechanism with stage 2 containers arranged in a circle. The stage 1 container as seen in Fig. 15 is below this carousel closer to the pan.

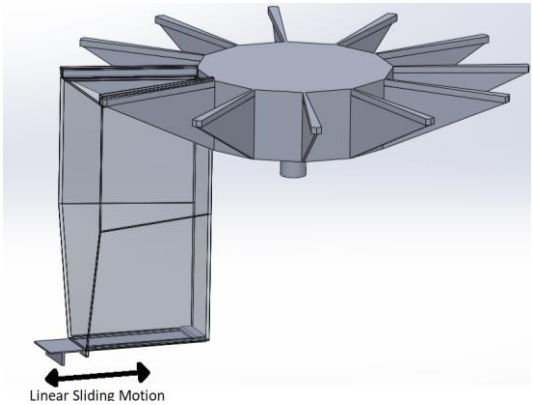


Fig. 13. CAD model of Stage 2 container

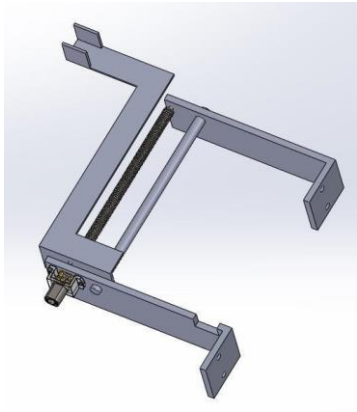


Fig. 14. CAD model of Stage 2 opener

Stage 2 (Fig. 13) contains 2 different types of containers with the same outer structure but one with greater opening area than other for dispensing of macro ingredients & spices respectively as required by the recipe. This stage is rotated by a 60 rpm DC motor (Motor 8). The angle of rotation is controlled by an encoder such that the positions of the containers are exactly above stage 1. Stage 2 has a lead screw opening mechanism for containers using a N20 motor (Motor 7) as shown in Fig. 14, to let ingredients fall into stage 1 where it is weighed using a weight sensor. This gave us the advantage of using only one mechanism for all the containers in Stage 2.

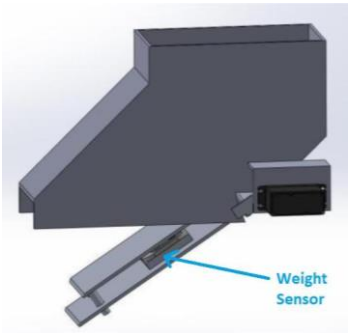


Fig. 15. CAD model of Stage 1 container

Stage 1 has a flap type opening mechanism that uses a servo motor which lets the weighed ingredients slide down in the pan. It also has the said weight sensor to receive the right amount of ingredients from stage 2.

Macro ingredients containers are expected to be refilled to the right amount before initiation of each cooking cycle, depending on the recipe.

3.4.2.2. Liquid Ingredient Dispenser and Semi-Liquid Ingredient Dispenser

The system contains one liquid dispenser and one semi-liquid dispenser. The liquid dispenser is for oil & the latter is for the Dosa Batter as shown in Fig. 16 & Fig. 17 respectively. For water, a hose is used which will be directly connected to a water supply. The dimension of the batter dispenser is decided to hold enough quantity for a meal of two people.

The dispensing action of all these fluids are controlled using a Solenoid Valve 1/2" DC 3.6-6V. The dispensing action is a gravity-based system for oil and batter.

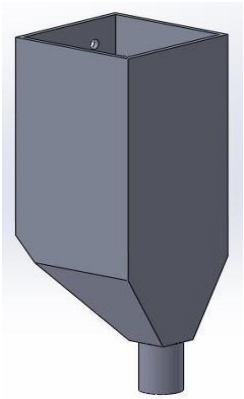


Fig. 16. CAD model of Oil dispenser

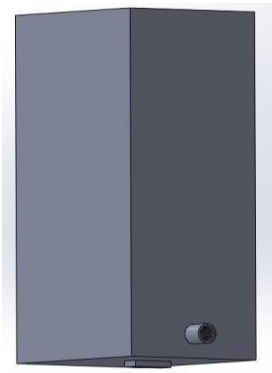


Fig. 17. CAD model of Batter dispenser

3.4.2.3. Actuators and Electronics

Actuators going to be used in the system include many motors and Solenoid Valves. Solenoid valve decided is Solenoid Valve 1/2" DC 3.6-6V Water Control Electric Pulse (Bistable)

The summary of all motors is tabulated in Table 2.

Table 2. List of Motors used in the model			
Motor Position	**Model**	**Specifications**	**Justification**
Motor 1	Orange 12V Brushed DC	10 RPM, 472.6 Ncm	Slow Speed for precision
Motor 2	Orange 12V Brushed DC	10 RPM, 472.6 Ncm	High Torque Required
Motor 3	Orange 12V Brushed DC	10 RPM, 472.6 Ncm	High Torque Required
Motor 4	Orange 12V Brushed DC	10 RPM, 328.3 Ncm	High Torque Required
Motor 5	Orange 12V Brushed DC	60 RPM, 157.6 Ncm	Medium Speed to retain precision with relatively lower torque requirement
Motor 6	N20 12V Micro Brushed DC	60 RPM, 9.5 Ncm	Compact size, Lower torque requirement
Motor 7	N20 12V Micro Brushed DC	600 RPM, 1.8 Ncm	Compact size, Lower torque requirement
Motor 8	Orange 12V Brushed DC	60 RPM, 157.6 Ncm	Medium Speed to retain precision with relatively lower torque requirement
Motor 9	TowerPro MG995 Plastic Gear Servo Motor	180º, approx. 19 Ncm, 50-62.5 RPM	Low Load, Easy to control

The control system for this includes an Arduino Mega 2560 Rev3 microcontroller which forms a feedback loop system using the inputs given by different encoders and sensors. The motors being considered are compatible with OE-28 Hall Effect Two Channel Magnetic Encoder which is an incremental encoder.

To measure the weight of ingredients as mentioned before, a 50kg Half-bridge Experiments Body Scale Load Cell Sensor is going to be used.

3.4.3. Special cooking tools

Specialized tools have been designed, with handles optimized specifically for being gripped by the claw. These special tools consist of a Stirring tool (Fig. 18) for stirring of gravy and different one pot meals, Flipping

tool (Fig. 19) for making food items such as *Dosa* and *Uttapa*, Mashing tool (Fig. 20) for making food such as *Bhaji* for *Pav Bhaji* and a Spreading tool for spreading batter for items such as *Dosa* (Fig. 21).

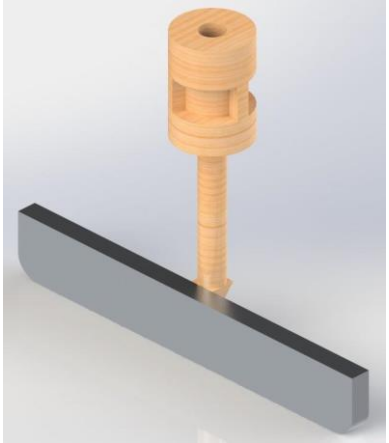


Fig. 18. Stirring tool

Fig. 19. Flipping tool

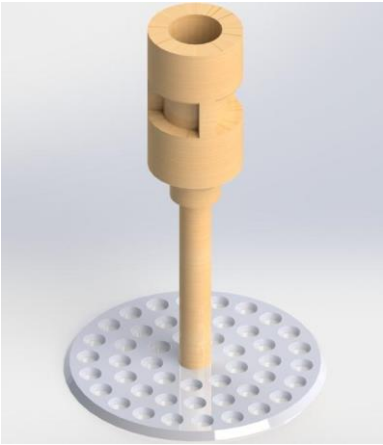


Fig. 20. Mashing tool

Fig. 21. Spreading tool

3.5. Interaction with Industry personnel

Once we had our detailed design for the system ready we approached certain experts in the field of designing and manufacturing, seeking assistance with design validation and manufacturing procedures. The first was Dr. Nilesh Raykar, a professor in the Mechanical department of SPCE and the other, the founder of Prasanna Packaging Machinery Pvt. Ltd., Mr Sanjiv Sarnaik along with their Head Design Engineer, Mr Pradip Gujare. They provided valuable inputs from an industrial perspective, like the scaling down of the design for ease of prototyping. Other valuable inputs included the suggestion to use 1mm thickness square cross section Aluminum hollow rod for link 1 and link 2 of the arm in place of 5mm thickness square cross section Nylon hollow rod which we were planning to use for the weight reduction as the former was more readily available. Along with these there were also some design changes suggested by them for ease of assembly and manufacturing and for avoiding certain instances of over engineering.

3.6. Design Modification for using standard parts

As mentioned before, initially it was decided that the stage 2 containers that store the solid ingredients will be custom made but later on owing to its complex structure it was found that it will be infeasible to manufacture them in low quantity. For these reasons different alternatives were explored and finally it was decided that we will use readymade plastic containers. After deciding upon the exact containers; the design of the dispenser stand was also modified accordingly. Old container along with the new container can be seen side by side in the figures below.

Fig. 22. Custom made dispenser

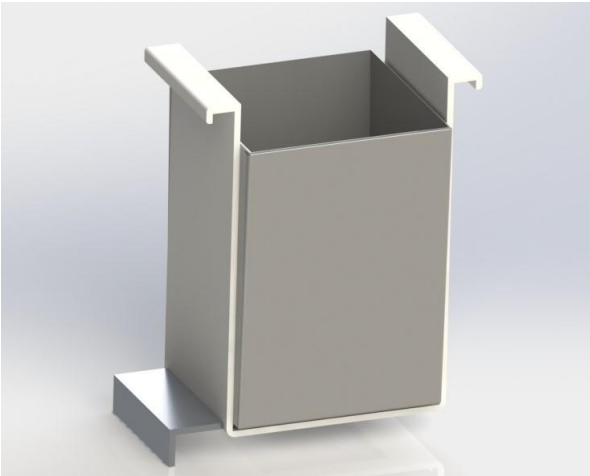


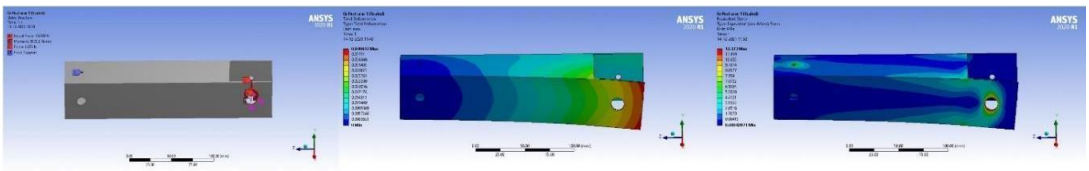
Fig. 23. Market dispenser

Chapter 4. Analysis

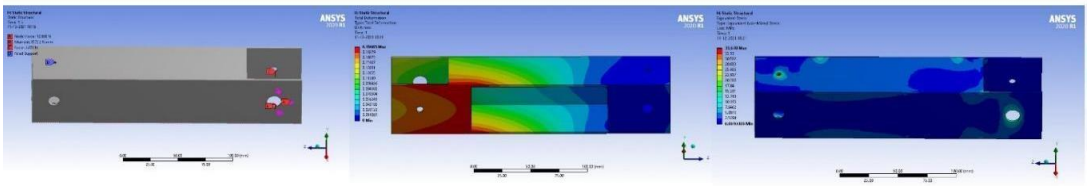
Analysis is carried out using Ansys [13] to verify the structural and thermal integrity of all materials and components used in the design.

4.1. Structural Analysis

Static Structural analysis of the links 1 and 2 with material as aluminum was carried out using ANSYS. Since the deformation of the link was very less & also in order to reduce the weight of the arm which would further reduce torque on the motor, geometrical changes to the cross-section were made & analyzed. The first variant had a cut through one of its sections whereas the second variant was a truss like structure as shown in Fig. 24 & Fig. 25. These changes did not satisfactorily reduce weight thus for the material of the links other non-metallic materials were also considered. Precedent for usage of polyamide (Nylon 66) as an alternative was found in the analysis by Kaitwanidvilai et al [14]. On conducting the analysis for Nylon, a satisfactory value for weight, deformation and stress was achieved but after further considerations for prototyping purpose Nylon was rejected as it wasn't an economical option. Thus, we continued with aluminum but reduced the cross-section thickness to 1mm. Upon analysis, the weight reduction was better than all aluminum designs & deformation results were satisfactory even with the truss type cuts in the structure as shown.



Design 1



Design 2

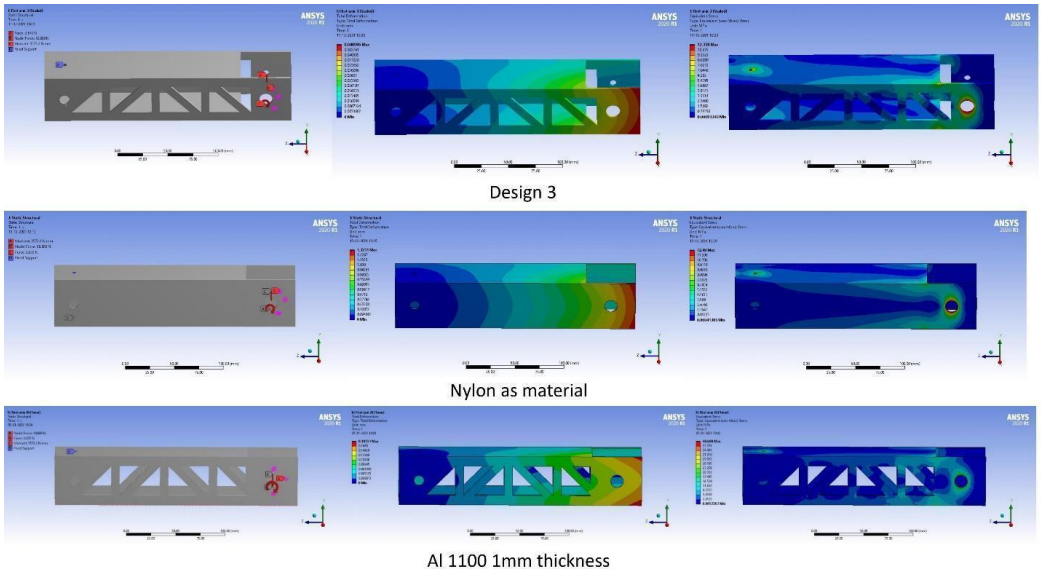


Fig. 24. Structural Analysis Results for iterations of First Link of the Robot Arm

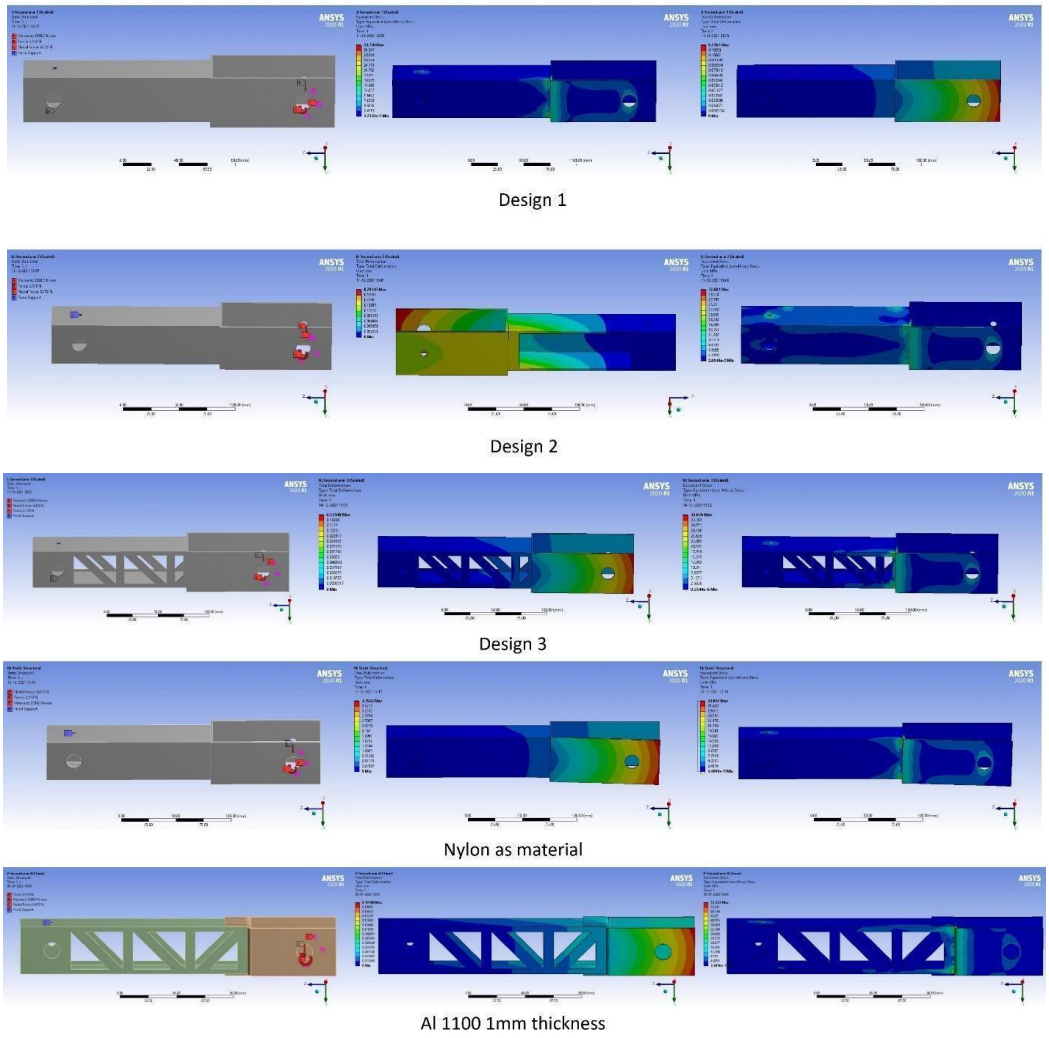


Fig. 25. Structural Analysis Results for iterations of Second Link of the Robot Arm

4.2 Thermal Analysis

In order to make sure that the heat from the induction cooktop does not affect the working of the motors by heat transfer, transient thermal analysis is done. Transient thermal analysis is chosen as the operation here varies with time & is not constant. A constant heat flux equivalent to the power corresponding to the maximum temperature of the cooktop is applied throughout the whole analysis. The ambient temperature of the surrounding area was taken as 30°C. The initial temperature of the setup was equal to the ambient temperature. Since radiation and convection will work towards decreasing the temperature of the whole system, only conduction was considered for the worst-case scenario and to simplify the analysis. Appropriate materials were assigned to each component. The result of the analysis is as shown in Fig. 26. Due to the material of the stirrer handle being nylon with low thermal conductivity, heat is prevented from going further thus reducing the effect of heat transfer in the claw & thereby motors attached to it.

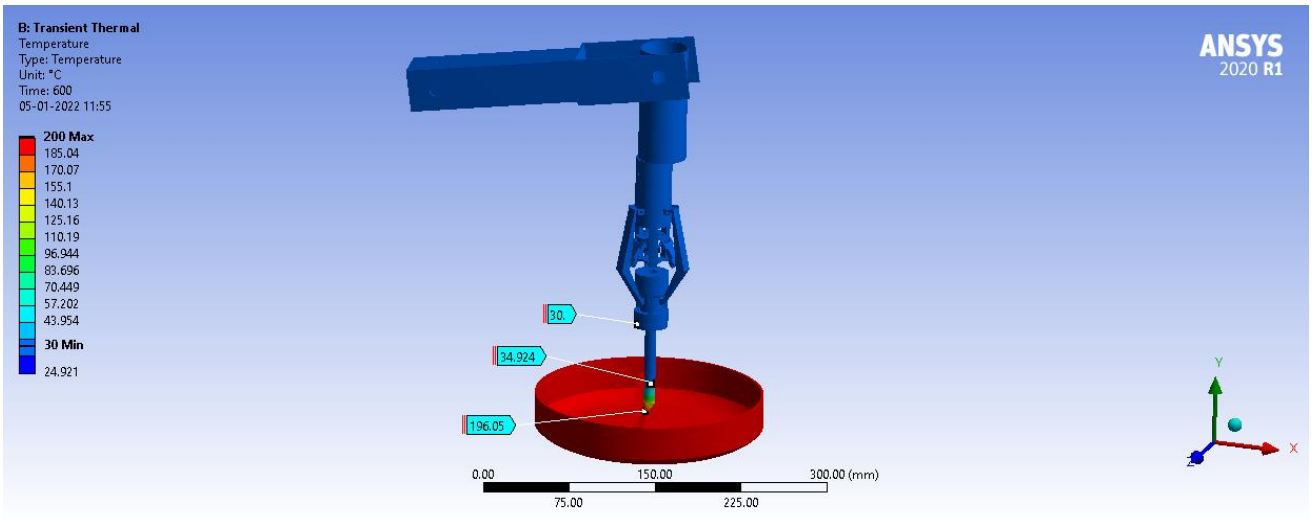


Fig. 26. Thermal Analysis Result of the model

4.2.1. Validation of Thermal Analysis

The methodology for this was validated by doing a similar experiment, an aluminum C-block and a strip of Delrin were used in the setup as seen in Fig. 27 & Fig. 29 (a). They were kept on a hot plate whose max power output is 500W. Readings for temperature near the separation of aluminum C-block and Delrin plate and time were recorded for every 10 °C rise of hot plate.

Fig. 27. Setup for Thermal Analysis Validation

The thermometer used for measuring the hot plate temperature has a range from 0 °C to 360 °C. The thermometer used for measuring the separation temperature ranges from 0 °C to 150 °C. The data recorded is seen in Table 3 and Fig. 28.

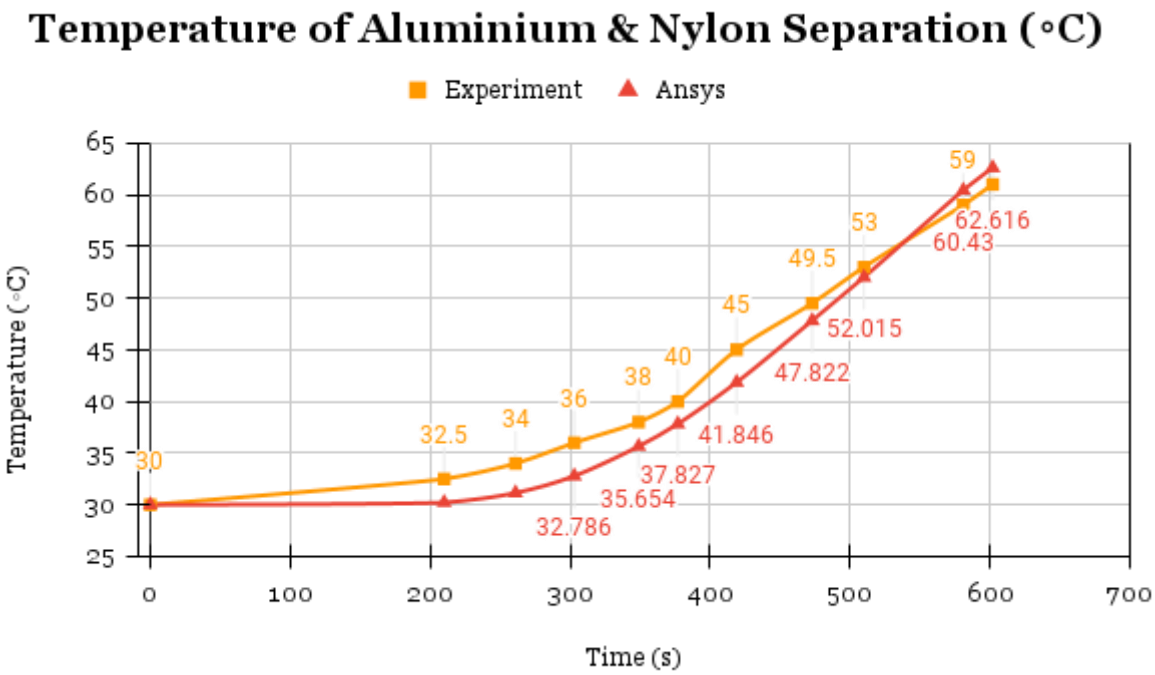
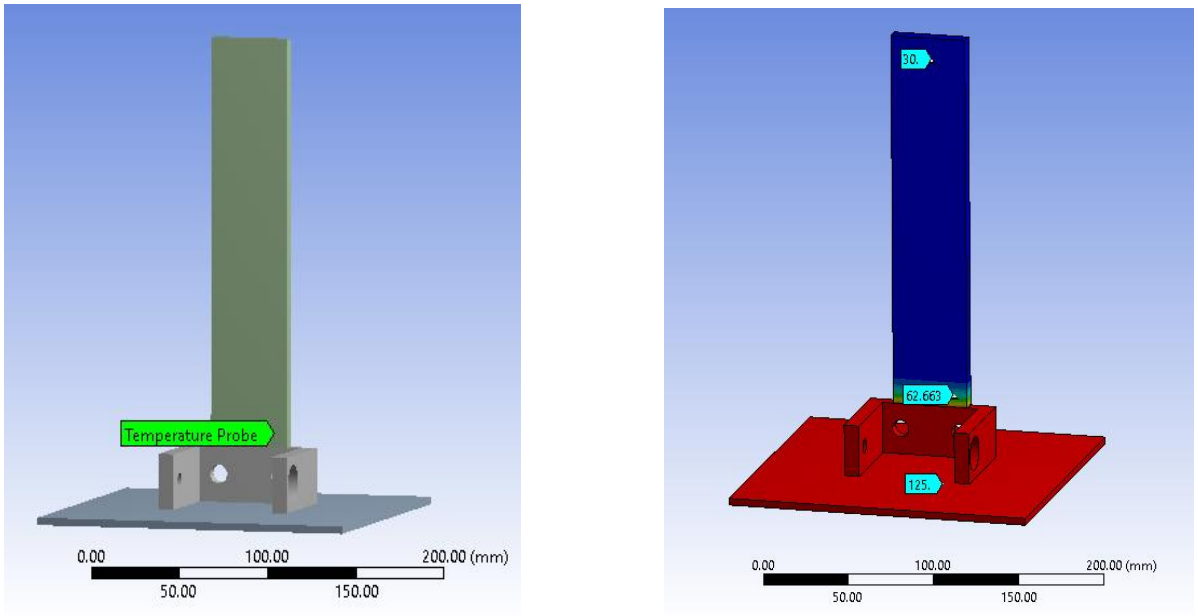


Fig. 28. Graph for Experimental vs Ansys observations for Thermal Analysis Validation

Table 3. Experimental observations for Thermal Analysis Validation				
Temperature of the Hot Plate (◦C)	**Time (s)**	**Temperature at the separation of Aluminum & Delrin (°C)-Experiment**	**Temperature at separation of Aluminum & Delrin (°C)-Ansys**	**% Error =** $\frac{(Ansys-Experiment)*100}{Experiment}$
30	0	30	30	0
40	210	32.5	30.218	-7.021538462
50	261	34	31.149	-8.385294118
60	303	36	32.786	-8.927777778
70	349	38	35.654	-6.173684211
80	377	40	37.827	-5.4325
90	419	45	41.846	-7.008888889
100	473	49.5	47.822	-3.38989899
110	510	53	52.015	-1.858490566
120	581	59	60.43	2.423728814
125	600	61	62.616	2.649180328



(a) (b)

Fig. 29. (a) CAD Model of the Validation Setup (left); (b) Ansys Results of Thermal Analysis of the Validation Setup (right)

4.3. Simulation & Kinematic Analysis

A simulation of the robot arm was carried out using Cyberbotics WeBots™ for Windows to aid in the development of the control algorithm and the trajectory characteristics.

The algorithms are coded in Python and designed to allow the endpoint of the arm to attain positions corresponding to input coordinates. Interpolation between these points is carried out in joint space, independently for each joint based on its current and target angle, the latter of which is to be obtained by performing inverse kinematics. To ensure conformity to specific cartesian space curves, the number of points between the initial and final position can be increased.

4.3.1. Kinematic Analysis

Inverse kinematics refers to the conversion of data about the position of the end effector from the global cartesian space to the joint space; forward kinematics being the other way around [15]. Inverse kinematics was needed to translate between known information; i.e., the coordinates of the end effector must reach in order to carry out a certain cooking action and the unknown joint angles that would correspond to said coordinates. The Denavit-Hartenberg Representation [16] is the popular choice of representation of an inverse kinematic model for a robot manipulator [17-20]. However, the current robot model, due to its simplicity, can be conveniently modeled using explicit equations pertaining to its geometry. These are explained below.

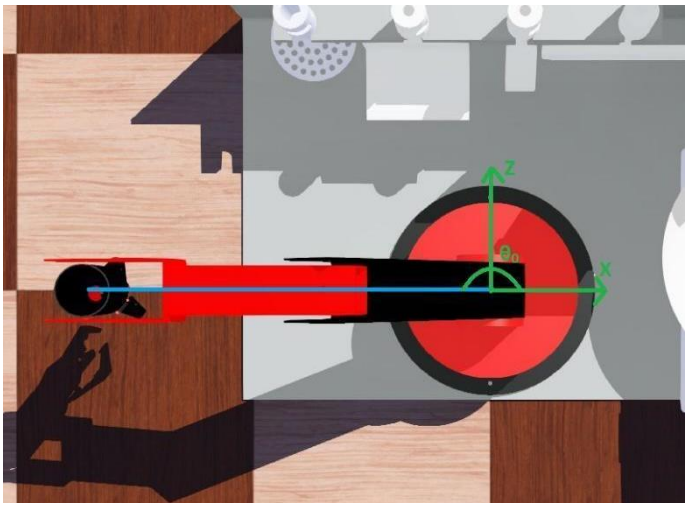


Fig. 30. Angle system for base

By considering the plane of the linkages to be movable about the vertical axis through the origin, analysis can be simplified.

First, based on coordinates (x,y,z) of the end effector, angle θ_0 of the plane from Fig. 30 is:

$$\theta_0 = arctan\left(\frac{z}{x}\right) \qquad where\ x > 0 \qquad \text{…..(17)}$$

$$\theta_0 = \pi - arctan\left(\frac{z}{x}\right) \qquad where\ x < 0 \qquad .....(18)$$

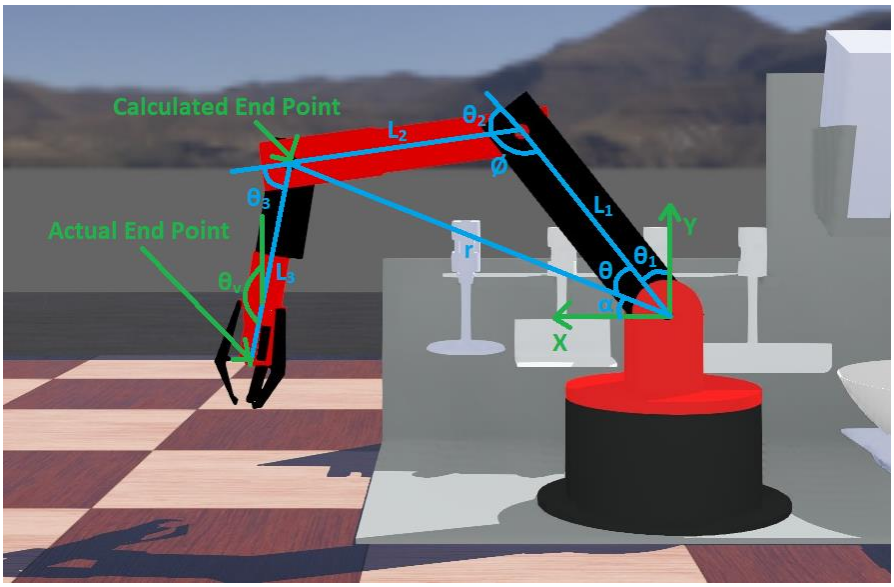


Fig. 31. Angle system for links in the rotating plane

Now, shifting from the global frame to the rotating vertical plane frame,

$$X_{act} = (x^2+z^2)^{0.5} \qquad .....(19)$$

However, the arm possesses 3 DOF (excluding the wrist rotation and claw position) in this plane, leaving 1 DOF redundant since points in a plane possess only 2 DOF. Hence, an additional constraint for the arm position must be provided, to arrive at a finite number of solutions. This is chosen to be θ_V i.e., the angle between link 3 and the vertical, which will be known for the various actions the arm will carry out.

Additionally, to accommodate the new angle constraint, the actual end point is changed to the calculated end point as shown in Fig. 31.

Hence, for this point,

$$X = X_{act} + L_3 * cos\left(\frac{3\pi}{2} - \theta_V\right) \qquad .....(20)$$

$$Y = Y_{act} + L_3 * sin\left(\frac{3\pi}{2} - \theta_V\right) \qquad .....(21)$$

$$\alpha = arctan\left(\frac{Y}{X}\right) \qquad where\ X > 0$$

$$\alpha = \pi + arctan\left(\frac{Y}{X}\right) \qquad where\ X < 0 \qquad .....(22)$$

$$\theta = arccos\left(\frac{\left(L_1^2+r^2-L_2^2\right)}{2L_1 r}\right) \qquad .....(23)$$

$$\phi = arccos\left(\frac{\left(L_1^2+L_2^2-r^2\right)}{2L_1 L_2}\right) \qquad .....(24)$$

Therefore, θ_1, θ_2 and θ_3:

$$\theta_1 = \left(\frac{\pi}{2}\right) - (\theta + \alpha) \quad \text{.....(25)}$$

$$\theta_2 = \pi - \phi \quad \text{.....(26)}$$

$$\theta_3 = \theta_v - \theta_1 - \theta_2 \quad \text{.....(27)}$$

Where,

Li=Length of Link i

Xact= Abscissa of actual endpoint of the arm acc to the rotating plane axes

Yact= Ordinate of actual endpoint of the arm acc to the rotating plane axes

X= Abscissa of endpoint of the link 2 (calc endpoint) acc to the rotating plane axes

Y= Ordinate of endpoint of the link 2 (calc endpoint) acc to the rotating plane axes

4.3.2. Angular Velocity Interpolation Function

The velocity function is designed so as to create a smooth curve between the start and end points, avoiding sudden accelerations and decelerations considering the proposition by P. Gandhi et al [18]. Feedback is available in terms of time elapsed and current angle of the joint.

Hence, in the Position-Velocity Space, a squared trigonometric function is used, considering that all its derivatives are continuous. A temporary, linear time dependent term is added to the function to initiate velocity in a smooth manner; because at position=0, velocity must be 0, making it impossible for it to change with respect to time.

$$If\left(\frac{TargetAngle - CurrentAngle}{TargetAngle - StartAngle} > 0.8\right):$$

$$v = c_1 * (MaxSpeed) * sgn(TargetAngle - CurrentAngle) * sin^2\left(\pi \frac{CurrentAngle - StartAngle}{TargetAngle - StartAngle}\right) + c_2 * sgn(TargetAngle - CurrentAngle) * T \quad \text{.....(28)}$$

$$If\left(\frac{TargetAngle - CurrentAngle}{TargetAngle - StartAngle} \leq 0.8\right):$$

$$v = c_1 * (MaxSpeed) * sgn(TargetAngle - CurrentAngle) * sin^2\left(\pi \frac{CurrentAngle - StartAngle}{TargetAngle - StartAngle}\right) \quad \text{.....(29)}$$

Where,

c1 and c2 are arbitrary scaling constants.

Chapter 5. Fabrication and Testing

5.1. Material Selection

We divided the entire design into 4 parts and then allotted materials to its sub-parts depending upon the manufacturability of the design and strength requirements as shown in Table 4.

Table 4. Materials table

Part	Sub-Part	Material
Robot Arm	Base	Mild Steel
	Link 1, 2 & 3	Aluminium 1100
	Motor Couplers	Delrin
	Potentiometer holders & couplers	ABS
Claw	Fingers & Links	Aluminium 1100
	Motor House	ABS
Ingredient Dispenser	Stand	Mild Steel
	Stage 1 Container	Aluminium 1100
	Stage 2 Container	Plastic - PET
	Hanger	Mild Steel
	Sliding Lid	Aluminium 1100
	Motor Stand	Aluminium 1100
	Dispenser Flap	Aluminium 1100
Miscellaneous	Tool Holder	Mild Steel
	Oil & Batter Container	Aluminium 1100
	Frame	Mild Steel
	Tool Handles	Delrin

5.2. Manufacturing Methodology

A variety of manufacturing and fabrication techniques had to be applied to build the necessary components of the robot.

These include:

- **Turning**
- **Milling**
- **Waterjet Cutting**
- **Bending**
- **Welding**
- **3D Printing**

First the process began with making 2D sheets for each component that had to be manufactured as seen in Fig. 32 and were taken to manufacturers for validation. After incorporating the changes suggested by them, some in design and some in creating 2D sheets the actual manufacturing process began.

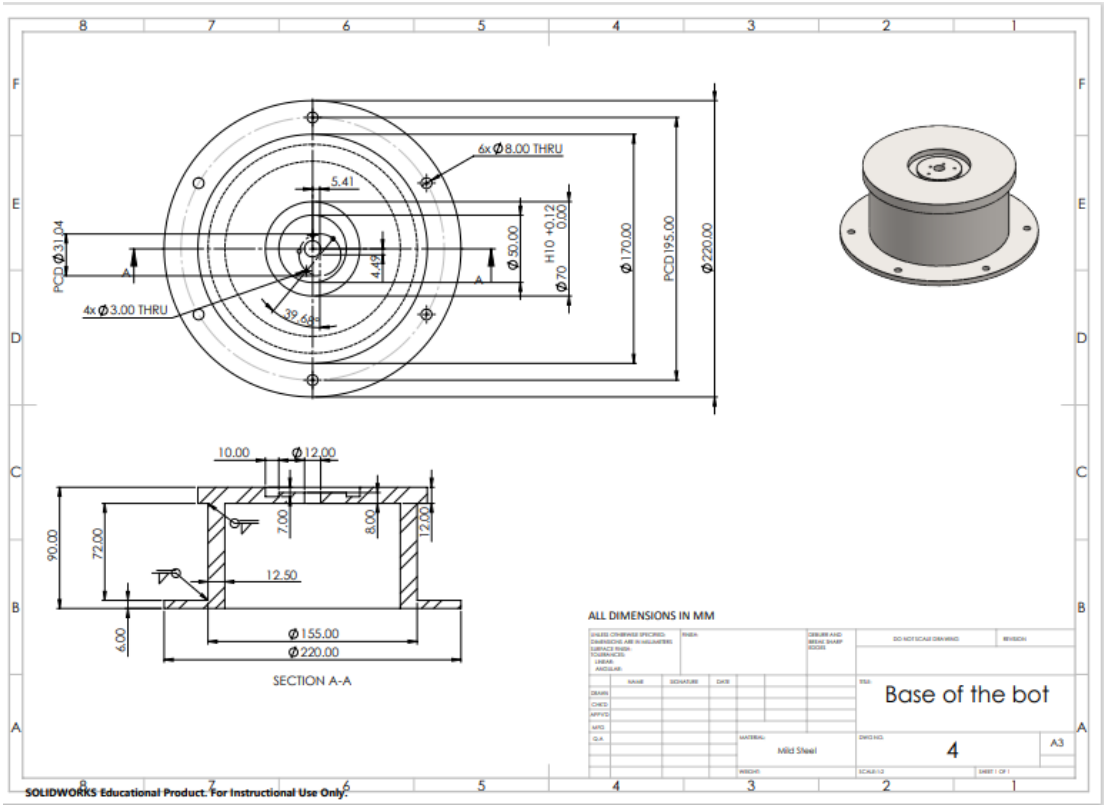


Fig. 32. Example of 2D designs showed to vendors

Lathe machining was preferred over milling for circular parts like the manipulator base, rotating base, motor house and motor couplers.

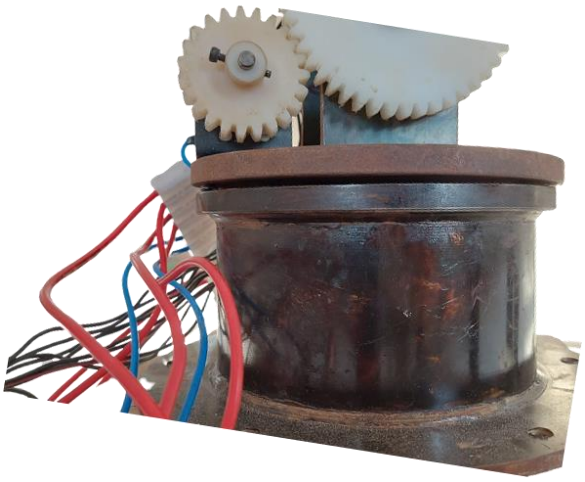


Fig. 33. Actual Base of the Manipulator

Welding had to be used for the base. It is worth noting that this process contributed to our knowledge as design engineers by helping us learn that designing a male-female fit into sections that need to be welded together greatly increases the ease and quality of welding. We can see the welded base of the arm in Fig. 33.

For intricate and unusual geometrical parts like fingers of the claw and sheet metals as seen in Fig. 34. We first decided to go for a waterjet. After getting some of these parts manufactured, based on recommendations by the manufacturer, laser cutting was prioritized for the remaining parts as it was cheaper and more precise.

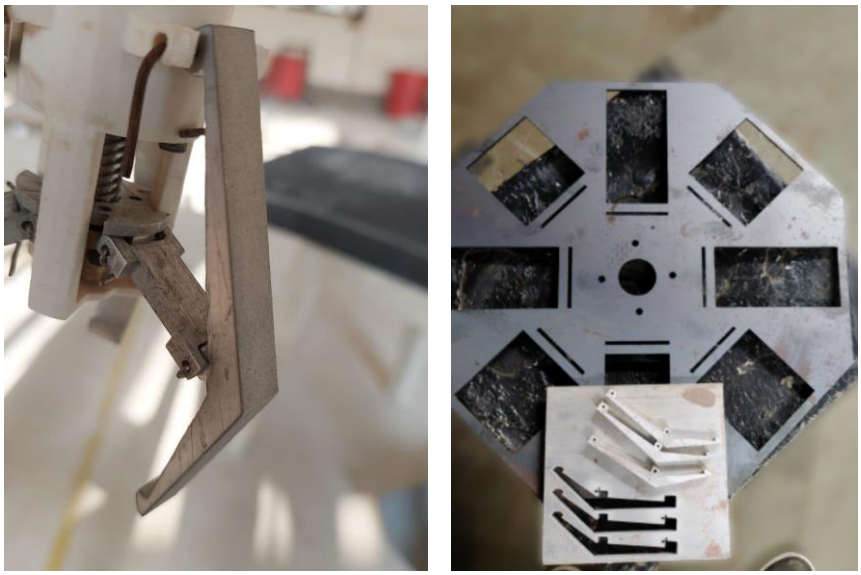


Fig. 34. (a) Fingers of the Claw after Waterjet Cutting (left); (b) Dispense Hanger after Laserjet Cutting (right)

Certain parts like the end attachment for Link 2 (Fig. 35) were manufactured by applying folds through a bending process to an appropriately shaped planar aluminum sheet.

The parts which didn’t need much strength and were too complex to be made using normal processes were fabricated additively using 3D Printing, thus saving us some time and cost. Use of this manufacturing

technique also taught us about the design features required for effective 3D printing such as support structures and about the types of geometries that can and cannot be 3D printed. Simple operations like drilling, cutting and some turning were carried out within the college on our own to minimise cost.

Fig. 35. First & Second Manipulator Link

5.3. Standard parts

Following is the list (Table 5) of readily available standard parts which influenced the design of our system.

Table 5. Standard Parts

Parts	Specification	Quantity
Bearings	SKF 51104	1
	SKF 51110	1
	SKF 61800	2
Bolts	M3	40
	M4	16
	M5	15
	M6	1
	M8	6
Shafts	10mm dia	1
	6mm dia	2
Leadscrew	200mm Trapezoidal 4 Start Lead Screw 8mm Thread 2mm Pitch	1

Along with these standard parts, some standard plastic containers (Fig. 36) were also bought for use as stage 2 in the dispenser assembly as mentioned previously.

Fig. 36. Standard Plastic containers

5.4. Electronics

After manufacturing we proceeded with the development of the electronics & Control system. The first step was to find a suitable ac to dc converter for which we chose SMPS. The selection of SMPS was done based on the total current & voltage required by all actuators & sensors in the system which came out to be 20A when all motors work simultaneously at max load, also adding an FOS of 1.5 to the current. Since the motors

needed 12V supply the SMPS we selected has a 12V DC output. The microcontroller being used (Arduino Mega) needs a 5V supply and a Buck circuit using IC 7805 voltage regulator which took the 12V from the SMPS & regulated it to 5V was used to supply the same. The whole circuit can be seen in Fig. 37.

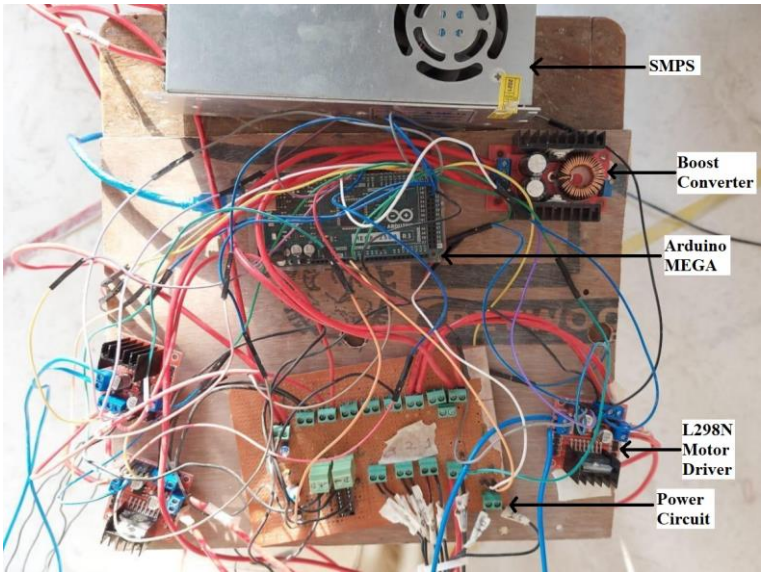


Fig. 37. Complete electronic circuit

The DC motors are driven by L298N motor drivers Fig. 38 which upon receiving input from the arduino, amplify the voltage to run the motors.

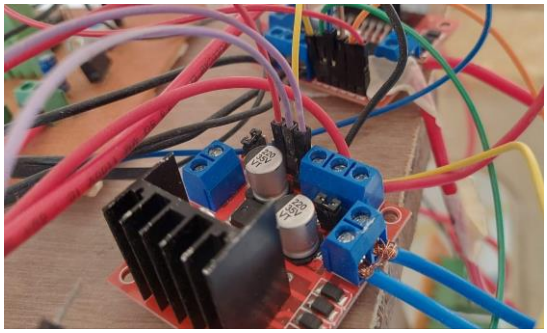


Fig. 38. L298N Motor Driver

Arguably the most important part in the electronics domain was the selection and configuration of sensors. Based on certain calculations, it was found that an accuracy and resolution of 1 degree was required at each joint. Among the sensors with this required accuracy, a 10 turn precision potentiometer Fig. 39 (a) was the only absolute measurement device that was within budget. However, since it could rotate through 10 turns and our design could only make use of 1 turn, any voltage supplied to the pot effectively became 1/10th for sensing purposes. To resolve this issue, some amount of testing was required and a detailed explanation for the same can be found in Section 5.6.

After testing, a special boost circuit (Fig. 39 (b)) was added to supply 50V to the potentiometer and it was mechanically configured so as to rotate through 270 degrees only by setting the voltage at 0 degrees as 2.5V. This serves to prevent the Arduino from receiving >5V which may damage it.

Of the 4 joints, 3 are controlled using these potentiometers which act as absolute encoders. The remaining one (rotation of base) is controlled using a hall effect incremental encoder due to space constraints in the design.

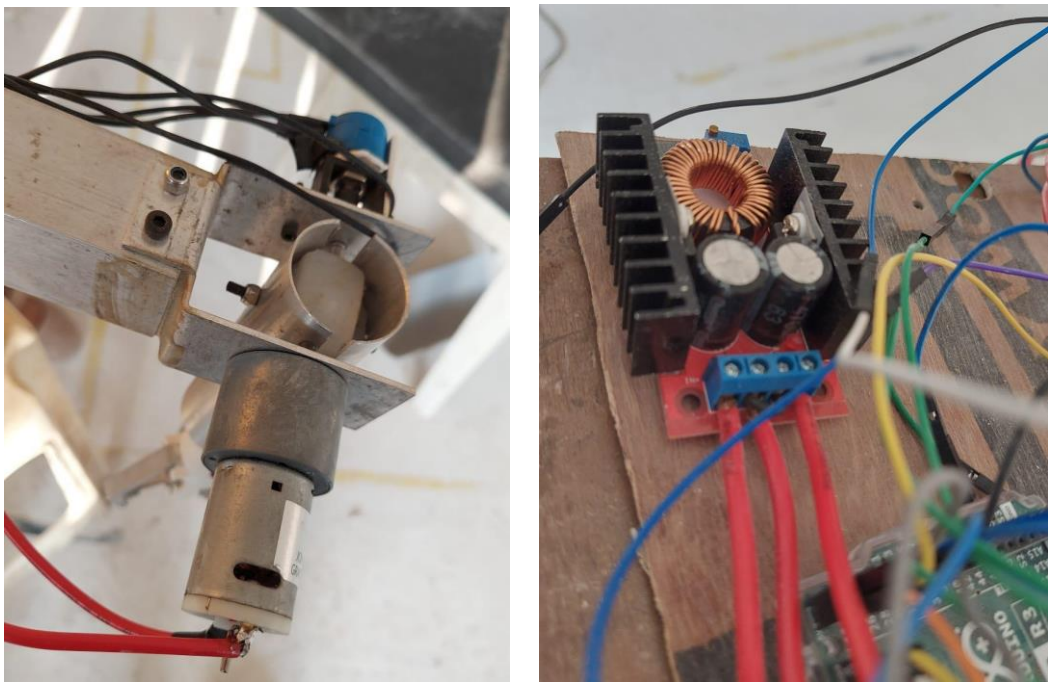


Fig. 39. (a) Potentiometer assembly (left); (b) Boost circuit (right)

5.5. The Code

The code for controlling the arm is essentially used to implement the Inverse Kinematic equations and velocity interpolation function mentioned in Section 4.3

It has been designed to be highly modular in order to expedite the testing process by allowing us to run nearly any subset of the functions at a given time and test their compatibility.

5.5.1. High Level Description

From a bird's eye view, the robot arm basically travels from Point to Point (PTP) and carries out certain actions at each one (like opening or closing the claw or rotating the wrist). Therefore, the fundamental input variable for controlling the robot arm is the set of points it needs to achieve in order to complete the required task, whatever it may be.

Each point consists of 4 values, the first 3 being the cartesian coordinates the end effector must reach and the 4th representing the orientation of the end effector in terms of angle with respect to the vertical at that point. Hence, the variable corresponding to the said set of points can be represented as a 2D Matrix (n*4) with

each row representing the point and the number of rows (n) representing the number of points it must achieve for completing the given task

Note that in the Arduino code, rather than an array based implementation of matrices, vectors are used because the latter can undergo dynamic size allocation, providing greater flexibility. To use these, the Arduino AVRSTL library needs to be included. The simulation code written in Python uses standard Python lists for the same purpose.

Once the points have been assigned, the code converts the cartesian coordinates into corresponding joint angles using inverse kinematic equations.

Then, it sends a signal to run each motor (PWM in the physical model and direct velocity signal for the simulation) till each joint attains the required angles one by one before finally coming to a stop. Feedback is obtained from the position sensors and is converted into joint angles for checking the difference between current and target angles.

Fig. 40. On the next page shows a flowchart for a high level view of the code.

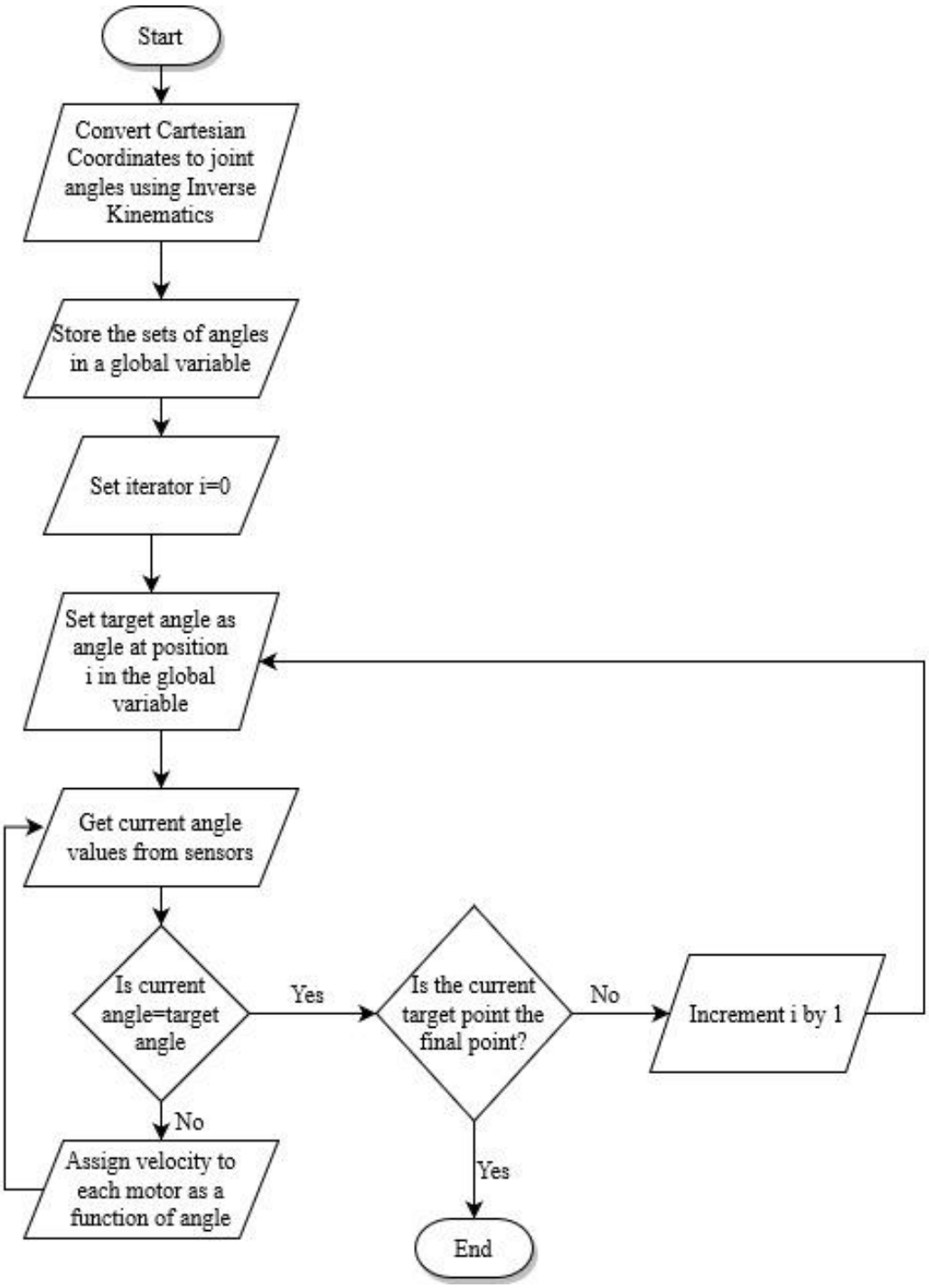


Fig. 40. High Level Flowchart of the code

5.5.2. Description of Important Features, Functions and Variables

Below is a comprehensive list of all the salient features of the code including functions and variables. The code used for simulation is logically identical and therefore, this report will only discuss the practically implemented Arduino code.

5.5.2.1 Important Variables

i. **cartcoor:** Used to store the set of points the end effector must achieve to complete a particular task. It is a vector of type float vector, i.e. it is a vector of vectors. Each vector it contains possesses four elements (as explained in section 5.5.1) and it may contain any number of such vectors. It is a global variable that starts out with a null value and is assigned a new value based on the task being carried out, using a special function.

ii. **ptset:** Used to store the set of joint angles the robot must achieve to complete a particular task. It is also a vector of type float vector, i.e. it is a vector of vectors. Each vector it contains possesses four elements (corresponding to the joint angles of the base, link 1, link 2 and end effector, each with respect to the previous link) and it may contain any number of such vectors. It is a global variable that starts out with a null value and is assigned a new value based on the task being carried out, in the **ikin** function by applying the inverse kinematic equations to every point in **cartcoor**.

iii. **count1:** Long type variable used to store the encoder pulse counts

iv. **offset:** A double type array used to store the offset in sensor readings used for error correction through linear regression

v. **mfactor:** A double type array used to store the proportionality constants for sensor reading correction through linear regression

vi. **cPara:** A n*2 2 dimensional vector that stores information for running the claw. Each row consists of 2 elements, one corresponding to the direction in which the claw motor is to be run and the other to the time for which it is to be run (in milliseconds)

vii. **s, m, d:** These are short type variables used to store the number of sensors, motors (not including the claw) and digital pins for controlling the motor directions

5.5.2.2. Important Functions

i. **ikin():** This function is used to convert the cartesian coordinates into joint angles. It accepts a cartesian coordinate vector as input and returns a vector containing the 4 required angles.

The function in its entirety can be seen in Fig. 41.

```
vector <double> ikin(vector <double> &cart)
{
  double x = cart[0];
  double y = cart[1];
  double z = cart[2];
  double thetav = cart[3];
  double theta0;
  double theta1;
  double theta2;
  double theta3;
  double theta;
  if (x > 0)
  {
    theta0 =  atan(z / x);
  }
  else
  {
    theta0 = pi + atan(z / x);
  }
  double xz = pow((pow(x, 2) + pow(z, 2)), 0.5);
  y = y - h;
  double X = xz + l3 * cos((3 * pi / 2) - thetav); //+ (pf == -1) * (7);
  double Y = y + l3 * sin((3 * pi / 2) - thetav);
  double alpha;
  if (X > 0){
    alpha = atan(Y / X);
  }
  else{
    alpha = pi + atan(Y / X);
  }
  double r = pow((pow(X, 2) + pow(Y, 2)), 0.5);
  double l1r = 2 * l1 * r;
  theta = acos((pow(l1, 2) + pow(r, 2) - pow(l2, 2)) / (l1r));
  theta1 = ((pi) / 2) - (theta + alpha);
  double l1l2 = 2 * l1 * l2;
  double phi = acos((pow(l1, 2) + pow(l2, 2) - pow(r, 2)) / (l1l2));
  theta2 = pi - phi;
  theta3 = thetav - theta1 - theta2;

  theta0 = ((180 * theta0) / pi);
  theta1 = ((180 * theta1) / pi);
  theta2 = ((180 * theta2) / pi);
  theta3 = ((180 * theta3) / pi);

  vector <double> res = {theta1, theta2, theta3, theta0};
  return res ;
```

Fig. 41. ikin function

ii. **trajfunc():** This function assigns velocity to each motor using the sensor readings as input. It remains active until the cumulative error between current and target values over all the joints crosses a minimum threshold. It also has provisions to optimize the rotation of the base by always allowing it to take the direction of rotation that has a smaller difference in values. Fig. 42 shows the function in its entirety.

```
vector <double> trajfunc(vector <double> &cpt, vector <double> &fpt, vector <double> &ipt, double start)
{
  double T = (millis() - start) / 1000;

  vector <double> velset = {};
  for (int i = 0; i < (pwmPins.size()); i++)
  {
    double thetadiff = fpt[i] - cpt[i];
    if (i == 3 && abs(thetadiff) > 180)
    {
      if (thetadiff > 0)
      {
        fpt[i] = fpt[i] - 2 * 180;
      }
      else
      {
        fpt[i] = fpt[i] + 2 * 180;
      }
    }
    double v = ((fpt[i] - cpt[i]) / abs(fpt[i] - cpt[i])) * (((fpt[i] - cpt[i]) / abs(fpt[i] - cpt[i]) > 0) * (vdw + (i == 2) * 10
    + ((fpt[i] - cpt[i]) / abs(fpt[i] - cpt[i]) < 0) * (vup));
    velset.push_back(v);
  }
  for (int i = 0; i < velset.size(); i++)
  {
    Serial.print("Motor ");
    Serial.print(i);
    Serial.print(": ");
    Serial.println(velset[i]);

  }

  return velset;
}
```

Fig. 42. trajfunc function

iii. **checkpulse1():** This is an interrupt sequence used to record encoder pulses from the base motor and store them using the **count** variable. It uses the relative phase between the A and B wave inputs from the encoder to deduce the sense of rotation and based on this information, either adds to or subtracts from the count. The interrupt is triggered when a rising wave front is detected from the A pulse waveform. It does not accept orreturn any values and only updates the global encoder count

The entire function can be seen in Fig. 43 below

```
void checkpulse1()
{
  if (digitalRead(encA1) == HIGH)
  {
    if (digitalRead(encB1) == HIGH)
    {
      count1++;
    }
    else
    {
      count1--;
    }
  }
  //Serial.println(count1);
}
```

Fig. 43. checkpulse1 function

iv. **sRead():** Takes readings from Arduino input pins connected to the potentiometers used to measure joint angles. It also converts the analog values and encoder count into angles and returns them as a vector after applying the regression based correction factors to the readings (See Section 5.5.3 for details)

The entire function can be seen in Fig. 44 below

```
vector <double> sRead() {
  vector<double> sReads = {};
  for (int i = 0; i < s; i++) {
    double sum = 0;
    for (int j = 0; j < 5; j++) {
      sum += (2 * 180 * (analogRead(ps[i]) - 512.0) / 1024.0);
    }

    sum = (mfactor[i] * (sum / 5) + offset[i]);


    sReads.push_back((2 * (i == 0) - 1)*sum);
  }
  sReads.push_back(count1 * 2 * 1800 / (7.0 * 18000.0));

  return sReads;
}
```

Fig. 44. sRead function

v. **command_motor():** It accepts a vector as input which is expected to contain the PWM values that need to be assigned to the 4 joint motors. It corrects for any overflow errors (values exceeding the upper permitted threshold) and writes the corresponding PWM to the Arduino pins. The function can be seen in Fig. 45 below

```
void command_motor(vector <double> &fvel) {
  for (int i = 0; i < (DigiPins.size() / 2); i += 1) {
    //Serial.println(fvel[i]);
    if (fvel[i] > 0) {
      digitalWrite(DigiPins[i], HIGH);
      digitalWrite(DigiPins[i + 6], LOW);

    }
    else {
      digitalWrite(DigiPins[i], LOW);
      digitalWrite(DigiPins[i + 6], HIGH);
      fvel[i] = abs(fvel[i]);

    }

    if (fvel[i] > 60) {
      fvel[i] = 60;
    }
    analogWrite(pwmPins[i], fvel[i]);
  }
}
```

Fig. 45. command_motor function

vi. **diffmag():** This function returns the square of differences between each joint's current and target value as a vector. It accepts the target and current point as 2 vector inputs. The function can be seen below in Fig. 46.

```
vector<double> diffmag(vector <double> &cpt, vector <double> &fpt) {
  vector <double> diff = {};

  for (int i = 0; i < m; i++) {
    diff.push_back(pow((fpt[i] - cpt[i]), 2));
  }

  return diff;
}
```

Fig. 46. diffmag function

vii. **ipolate():** This function receives only the target point in terms of angles as input. It then calls **sRead()** and **trajfunc()** and passes the value of the former to the latter to allow for proper velocity assignment. It also receives the set of motor velocities from **trajfunc()** and passes them on to **command_motor().** It then passes the current sensor value and target sensor value to **diffmag()** to check whether the target angle has been reached. If not, these steps continue to run in a while loop until **diffmag()** returns sufficiently small joint angle errors. Once the point has been reached, it sets the motor velocities to 0 using **command_motor()** directly. This function also allows for effective error correction in the joints because, even if any joint angle overshoots its target value before the others are in position, corrections will keep occurring until all joints reach the target value

```
void ipolate(vector <double> &nxtpt) {
  long start = millis();
  vector <double> getVel = {};
  vector <double> sVals = {};
  vector <double> ipt = sRead();
  vector<double> diff = diffmag(ipt, nxtpt);

  //print(diff)
  while (diff[0] > 0.5 || diff[1] > 0.5 || diff[2] > 0.5 || diff[3] > 0.5 ) {
    sVals = sRead();
    getVel = trajfunc(sVals, nxtpt, ipt, start);
    command_motor(getVel);
    sVals = sRead();
    diff = diffmag(sVals, nxtpt);
  }

  command_motor(zerovel);
}
```

Fig. 47. ipolate function

viii. **procedure():** This function is used to assign values to the **cartcoor** and **cPara** variables according to the procedure we want to carry out and then convert each point in **cartcoor** to its corresponding joint angle by running calling **ikin()** in a loop and updating the value of **ptset.** This function is called in **loop().**

Fig. 48 shows the exact code.

```
void procedure()
{
  ptset = {};
  cPara = {{0,1650}, {1,1650}};
  cartcoor = {{305.00, -31.79, 0, pi}};
  for (int i = 0; i < cartcoor.size(); i++)
  {
    vector<double> ang = ikin(cartcoor[i]);
    ptset.push_back(ang);
  }

  //   for(int i=0;i<ptset.size();i++)
  //    {
  //      for(int j=0;j<ptset[i].size();j++)
  //      {
  //        Serial.println(ptset[i][j]);
  //      }
  //    }

}
```

Fig. 48. procedure function

ix. **runclaw():** This function is used to open or close the claw by supplying a high signal to its motor for a set amount of time. Since the claw does not have a corresponding position sensor, this special function is required to run it to the extent that is desired. The input to this function is a vector containing the time in milliseconds for which the claw is supposed to run and the direction the motor is supposed to turn in (corresponding to either opening or closing the claw). In operation, this value is supplied from the **loop()** function and is taken from the global **cPara** vector The code can be seen in Fig. 49.

```
void runclaw(vector<float> &pd)
{ int dir = pd[0];
  int t = pd[1];
  long start = millis();
  while ((millis() - start) < t)
  {
    Serial.println((millis() - start));
    if (dir == 0)
    {
      digitalWrite(26, LOW);
      digitalWrite(32, HIGH);
      analogWrite(6, cPow);
    }
    else
    {
      digitalWrite(26, HIGH);
      digitalWrite(32, LOW);
      analogWrite(6, cPow);
    }
  }
  analogWrite(6, 0);
}
```

Fig. 49. runclaw function

x. **setup():** This is the default setup function that is executed just once after the Arduino is switched on. It is used to set up all the pins and assign any interrupts. The exact code can be seen in Fig. 50.

```
void setup() {
  for (int i = 54; i < (54 + s); i++) {
    ps.push_back(i);
    pinMode(i, INPUT);
  }

  for (int i = 2; i < (m + 2); i++) {
    pwmPins.push_back(i);
    pinMode(i, OUTPUT);
  }

  for (int i = 22; i < (22 + d); i++) {
    DigiPins.push_back(i);
    pinMode((i), OUTPUT);
  }
  Serial.begin(115200);
  pinMode(encA1, INPUT);
  pinMode(encB1, INPUT);
  attachInterrupt(digitalPinToInterrupt(encA1), checkpulse1, RISING);

}
```

Fig. 50. setup function

xi. **loop():** This is the default loop function used in arduino that runs repeatedly after setup until the Arduino is switched off. In this code it is used to call the **procedure()** function and update the values of **ptset** and **cartcoor**. It then loops through every point in **ptset** and **cPara** and passes them one by one to **ipolate()** and **runclaw()** respectively as target values. Once the procedure is completed, it assigns 0 velocity to the motors. Provisions have been made, using a global flag to ensure that this function runs only once. Additionally, it is also ensured that the robot doesn't start until the sensors receive power. The code can be seen in Fig. 51.

```
void loop() {

  if ((flag == 1) && (analogRead(A0) > 250)) {
    delay(1000);
    procedure();

    for (int i = 0; i < ptset.size(); i++) {
      ipolate(ptset[i]);
      if(i==0){
        delay(1000);
      }
      runclaw(cPara[i]);

      delay(100);
    }

    flag = 0;
  }
  command_motor(zerovel);
}
```

Fig. 51. loop function

5.5.3. Practical Changes to the Code

The code written for the simulation had to be converted to be fit for use through the Arduino IDE for practical usage, beginning with converting the Python instructions to C++ and Java. The underlying logic mostly remained unchanged with the exception of the velocity control. In the software, the joint velocities could be controlled directly whereas using the Arduino, only the voltage supplied to the motors is within our control. However, the relationship between the two is non linear (load acts as an additional pseudo-variable and is a function of joint angles) and complex, requiring us to make several changes to the trajectory function to allow it to run correctly.

An offset and multiplicative factor also had to be added to the sensor reads, obtained through linear regression, to eliminate calibration errors. The readings used for calibration and their scatter plot is shown in Table 6 and Fig. 52 respectively

Table 6. Experimental observations for Error Correction					
Joint 1 (Angles in degrees)		Joint 2 (Angles in degrees)		Joint 3 (Angles in degrees)	
X (Actual Angle)	Y (Sensed Angle)	X (Actual Angle)	Y (Sensed Angle)	X (Actual Angle)	Y (Sensed Angle)
10	9	10	20	10	23
20	18	20	30	20	34
30	29	30	40	30	41
40	39	40	52	40	52
50	49	50	62	50	60
60	59	60	72	60	70

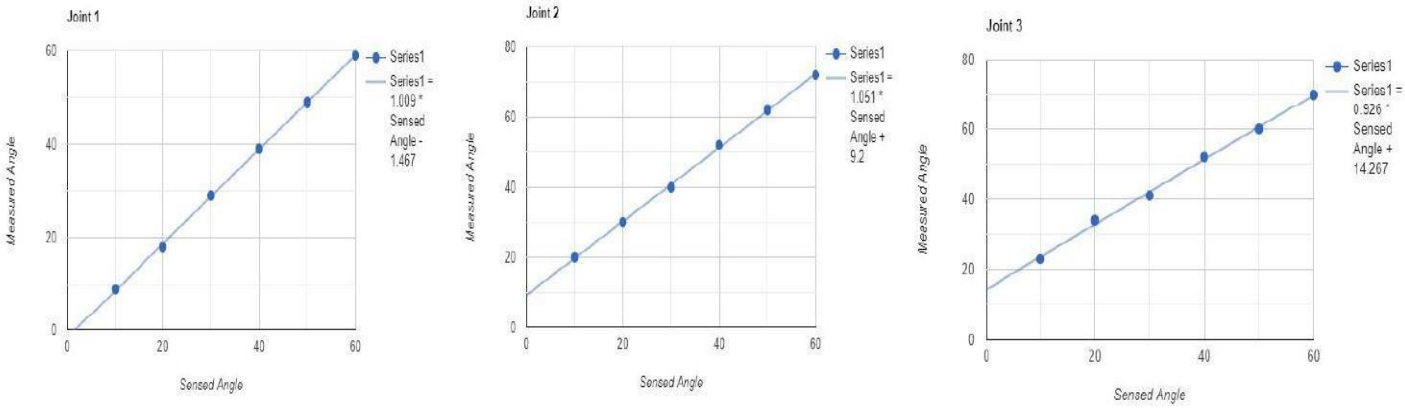


Fig. 52. (a) Scatter Plot for Joint 1; (b) Scatter Plot for Joint 2; (b) Scatter Plot for Joint 3

5.6 Testing

Fig. 53. Testing maximum load capacity of the arm with 0.7 kg motor

During the testing phase we started by testing the amount of precision that we were getting from the potentiometers. We began with a 5V supply that we had planned to use initially but upon testing we realized this would not result in sufficient accuracy. This is because the potentiometer we are using is a 10 turn potentiometer which means that the supplied voltage is linearly distributed over all the 10 turns. Our robot arm was only going to utilize the POT for a single turn. Thus the voltage difference that we could use was only 0.5V i.e. 0.1*supply voltage. Arduino has a 10 bit ADC for Input which converts voltage inputs to a 10 bit integer and it operates between 0-5V (0V corresponds to a reading of 0 and 5V to a reading of 1023). An input of 0.5V will, therefore, correspond to a reading of 102. This means that the effective resolution will be only 4° as 360°/102 is approximately 4. Thus the reading for degrees will be incremented in steps of almost 4°. To overcome this issue and get the required precision we decided to use a boost circuit to provide 50V to the POT which provided us with a resolution of approximately 0.36° by scaling up the voltage per turn from 0.5 to 5V.

Then we built and tested the power circuit before moving on to testing of the robotic arm assembly as a whole. Although we already had a final code ready, we did not test it directly for safety reasons. Capitalizing on the modularity of our code, which was a direct result of our design choices while developing the code, we

tested one function at a time. Once one was verified, the subsequent functions were tested one at a time, as independently as possible, making the debugging process significantly faster.

While testing the mechanical soundness of the design, we came to know that there is approximately 40 degrees of cumulative play in the joints due to the motors and, primarily, manufacturing defects. But the error correction provision in our code brought down the final error to 5 degrees overall.

After doing all these we finally tested our Inverse Kinematic function upon the success of which, we started hardcoding different processes for it to perform. As of now of the 6DOF, 5DOF are working completely. The wrist rotation motion of the claw is currently restricted due to unavailability of a slip ring to avoid entangling of the wire inside.

Chapter 6. Results and Discussion

6.1. Results & Conclusion

In this study, an automatic cooking system was designed with a 6 DOF robotic arm driven by brushed DC motors (with encoders), a rotating macro-ingredient dispenser assembly, and some other special dispensers. The robotic arm was analyzed for its structural strength. Thermal analysis was also carried out to ensure that there is no overheating of motors and temperature of materials is always below or within their serviceable limits. This analysis has been augmented by running kinematic and dynamic simulations of the system using softwares like Cyberbotics WeBots. This allowed us for research into optimizing the trajectory of the arm and estimating power consumption. A physical working model of the system has also been developed where the manipulator has been successfully tested. It can be seen in Fig. 54.

Practical development of this prototype has revealed certain merits and shortcomings in the design. First, the specific potentiometers used have high resolution and moderately high repeatability. This was concluded based on the fact that any errors were reducible to +-1° simply by applying linear regression between expected and measured values. Additionally, the viability of inexpensive brushed DC Motors for this application was verified, the only limitations being a relatively high response time and the non-linear nature of voltage based velocity control. The payload capacity was tested to be at least 42.7 kg-cm at the first joint (corresponding to a load of 0.7kg in the horizontal outstretched position). This project also helped highlight the importance of the philosophy of “Designing for Assembly”.

Overall this setup can be considered feasible for medium precision applications like cooking in terms of the purview defined in this report.

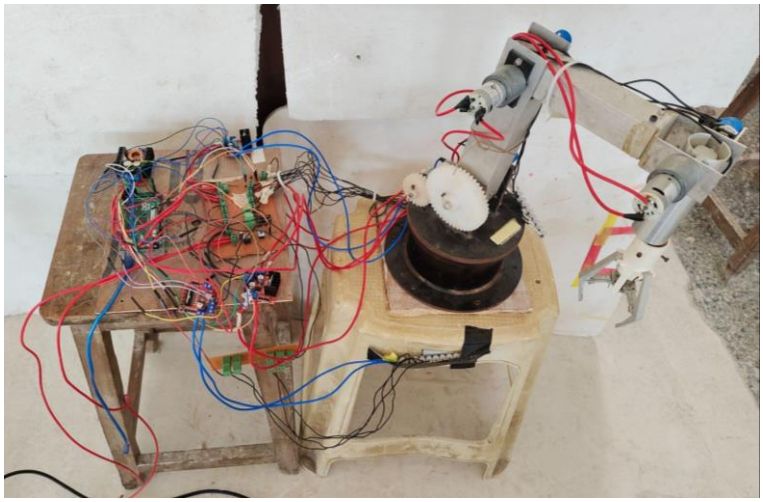


Fig. 54. Final Manufactured Robot Arm

It is hoped that this project will spur more research into the development of other such systems, overcoming some of its limitations while keeping the associated costs low.

6.2. Future Scope

This prototype design can be developed into a commercially viable automatic cooking system. It can further be improved in terms of mechanical design as well as in terms of its control system by integrating sensors like cameras for image processing, infrared cameras for remote temperature sensing, torque sensor, etc. Artificial intelligence & Machine learning can also be used to include more & more recipes or directly read a recipe from the internet [22]. The system can also be used remotely by leveraging IOT & using a mobile application [23]. Special dispensers for dispensing spices and seasoning with an auger style mechanism can be incorporated for higher precision.

Chapter 7. References

[1] Blöcher, K., & Alt, R. (2021). AI and robotics in the European restaurant sector: Assessing potentials for process innovation in a high-contact service industry. Electronic Markets, 31(3), 529-551.

[2] Zemke, D. M. V., Tang, J., Raab, C., & Kim, J. (2020). How to build a better robot... for quick-service restaurants. Journal of Hospitality & Tourism Research, 44(8), 1235-1269.

[3] Li, B., Chen, Y., Deng, Z., & Xu, W. (2011). Conceptual design and analysis of the 2T1R mechanism for a cooking robot. Robotics and Autonomous Systems, 59(2), 74-83.

[4] https://www.letsnosh.io/

[5] https://www.eatwithnymble.com/

[6] http://www.mechanicalchef.com/

[7] Moran, M. E. (2007). Evolution of robotic arms. Journal of robotic surgery, 1(2), 103-111.

[8] Wallén, J. (2008). The history of the industrial robot. Linköping University Electronic Press.

[9] Noh, D., Liu, Y., Rafeedi, F., Nam, H., Gillespie, K., Yi, J. S., ... & Hong, D. (2020, June). Minimal Degree of Freedom Dual-Arm Manipulation Platform with Coupling Body Joint for Diverse Cooking Tasks. In 2020 17th International Conference on Ubiquitous Robots (UR) (pp. 225-232). IEEE.

[10] Kirkpatrick, S., Gelatt Jr, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. science, 220(4598), 671-680.

[11] Kafuko, M., Singh, I., & Wanyama, T. (2015). Design of a Robotic Arm for Teaching Integrated Design. Proceedings of the Canadian Engineering Education Association (CEEA).

[12] Yue, H., Miao, J., Zhang, J., Fan, C., & Xu, D. (2021). Simulation for senior undergraduate education of robot engineering based on Webots. Computer Applications in Engineering Education, 29(5), 1176-1190.

[13] Kohnke, P. C. (1982). Ansys. In Finite Element Systems (pp. 19-25). Springer, Berlin, Heidelberg.

[14] Kaitwanidvilai, S., Buthgate, S., Aoyama, H., & Konghuayrob, P. (2020). Robot Arm Structure Design Using Polyamide Evaluated by Finite Element Analysis. Sensors and Materials, 32(2), 487-497.

[15] Kucuk, S., & Bingul, Z. (2006). Robot kinematics: Forward and inverse kinematics. INTECH Open Access Publisher.

[16] Denavit, J., & Hartenberg, R. S. (1955). A kinematic notation for lower-pair mechanisms based on matrices.

[17] Deshpande, V., & George, P. M. (2014). Kinematic modelling and analysis of 5 DOF robotic arm. International Journal of Robotics Research and Development (IJRRD), 4(2), 17-24.

[18] Xu, D., Calderon, C. A. A., Gan, J. Q., Hu, H., & Tan, M. (2005). An analysis of the inverse kinematics for a 5-DOF manipulator. International Journal of Automation and Computing, 2(2), 114-124.

[19] Tsumaki, Y., Kotera, S., Nenchev, D. N., & Uchiyama, M. (1997, April). Singularity-consistent inverse

kinematics of a 6-DOF manipulator with a non-spherical wrist. In Proceedings of International Conference on Robotics and Automation (Vol. 4, pp. 2980-2985). IEEE.

[20] Nie, L., & Huang, Q. (2012, December). Inverse kinematics for 6-DOF manipulator by the method of sequential retrieval. In the Proceedings of the International Conference on Mechanical Engineering and Material Science, China (pp. 255-258).

[21] Gandhi, P., Deshmukh, S., Ramtekkar, R., Bhole, K., & Baraki, A. (2013). " ON-AXIS" LINEAR FOCUSED SPOT SCANNING MICROSTEREOLITHOGRAPHY SYSTEM: OPTOMECHATRONIC DESIGN, ANALYSIS AND DEVELOPMENT. Journal of Advanced Manufacturing Systems, 12(01), 43-68.

[22] Bollini, M., Tellex, S., Thompson, T., Roy, N., & Rus, D. (2013). Interpreting and executing recipes with a cooking robot. In Experimental Robotics (pp. 481-495). Springer, Heidelberg.

[23] Livinsa, Z. M., Valantina, G. M., Premi, M. G., & Sheeba, G. M. (2021, March). A Modern Automatic Cooking Machine Using Arduino Mega and IOT. In Journal of Physics: Conference Series (Vol. 1770, No. 1, p. 012027). IOP Publishing.